1주차

OpenStack Networking — Chapter 1 & 3

Learning OpenStack Networking (Neutron) — James Denton, 1st Edition

---

Chapter 1. Preparing the Network for OpenStack

1. What is OpenStack Networking?

OpenStack Networking(Neutron)은 OpenStack 환경에서 가상 네트워크를 생성하고 관리하는 서비스이다.

전통적인 네트워크 환경에서는 관리자가 물리 스위치, 라우터, 방화벽 등을 직접 구성하여 네트워크를 관리했다.

반면 클라우드 환경에서는 사용자가 필요에 따라 VM과 네트워크를 생성하고 삭제하기 때문에, 물리 네트워크 장비를 매번 수동으로 구성하는 방식에는 한계가 있다.

이를 해결하기 위한 개념 중 하나가 **Software-Defined Networking(SDN)** 이다.

SDN은 네트워크 기능을 소프트웨어를 통해 추상화하고, 프로그래밍 가능한 방식으로 관리할 수 있도록 하는 네트워킹 접근 방식이다.

Neutron은 이러한 개념을 활용하여 사용자가 API를 통해 네트워크를 관리할 수 있도록 한다.

1.1 Neutron의 주요 역할

Neutron은 다음과 같은 네트워크 기능을 제공한다.

- 가상 네트워크 생성 및 관리
- VM의 네트워크 연결 관리
- 서로 다른 네트워크 사이의 라우팅
- 외부 네트워크 연결
- 네트워크 보안 정책 적용
- 추가적인 네트워크 서비스와 연동

Neutron은 다른 OpenStack 서비스와 마찬가지로 독립적인 서비스로 구성되며, 하나의 노드에서 실행하거나 여러 노드에 분산하여 배치할 수 있다.

주요 구성요소는 다음과 같다.

구성요소	역할
Neutron API	사용자의 네트워크 관리 요청을 받는 인터페이스
neutron-server	API 요청을 처리하고 네트워크 플러그인과 연결
Database	네트워크 구성 정보를 저장
Network Plugin	네트워크 기능을 특정 네트워크 기술로 구현하기 위한 연결 구조
Network Agent	네트워크 서비스 및 가상 네트워크 구성에 필요한 작업 수행

1.2 Neutron API

Neutron은 API를 통해 사용자가 네트워크를 생성하고 관리하도록 한다.

예를 들어 사용자가 새로운 가상 네트워크를 생성하면 다음과 같은 과정이 진행된다.

1. 사용자가 Neutron API에 네트워크 생성 요청을 전송한다.
2. neutron-server가 요청을 처리한다.
3. 생성된 네트워크 구성이 데이터베이스에 저장된다.
4. 선택한 Network Plugin과 관련 네트워크 구성요소가 필요한 연결을 구성한다.
5. Nova 등의 서비스가 생성된 네트워크를 이용하여 VM을 연결할 수 있다.

여기서 중요한 점은 **Neutron이 네트워크 구성을 관리하고, 실제 가상 네트워크 연결은 선택한 기술과 관련 구성요소를 통해 구현된다는 것**이다.

따라서 Neutron은 네트워크 API와 다양한 가상 네트워킹 기술을 연결하는 역할을 수행한다.

---

2. Features of OpenStack Networking

Neutron은 데이터센터에서 제공하는 여러 네트워크 기능을 가상화된 클라우드 환경에서 사용할 수 있도록 한다.

대표적인 기능은 다음과 같다.

- **Switching**
- **Routing**
- Load Balancing

- Firewalling
- Virtual Private Networks

2.1 Switching

Switching은 OSI 7계층 중 **Layer 2(Data Link Layer)** 에서 Ethernet Frame을 전달하는 기능이다.

일반적으로 스위치는 목적지 MAC 주소를 기준으로 프레임을 적절한 포트에 전달한다.

물리 네트워크에서는 하드웨어 스위치가 이 기능을 수행하지만, 가상화 환경에서는 소프트웨어 기반의 가상 스위치가 동일한 기능을 제공할 수 있다.

Virtual Switch

가상 스위치는 VM의 가상 네트워크 인터페이스들을 연결하는 소프트웨어 기반 네트워크 장치이다.

예를 들어 동일한 네트워크에 연결된 VM1과 VM2가 같은 Compute Node에 존재한다면 다음과 같은 경로로 통신할 수 있다.

```
VM1
  |
Virtual NIC
  |
Virtual Switch
  |
Virtual NIC
  |
VM2
```

VM1에서 발생한 Ethernet Frame은 Virtual Switch를 거쳐 VM2로 전달된다.

이 과정에서는 두 VM이 동일한 호스트에 있으므로 반드시 외부 물리 스위치를 거쳐야 하는 것은 아니다.

반면 서로 다른 Compute Node에 VM들이 존재한다면, 각 서버에 구성된 가상 스위치 사이에 네트워크 연결이 필요하다.

Linux Bridge

Linux Bridge는 Linux Kernel에서 제공하는 소프트웨어 기반 브리지이다.

여러 네트워크 인터페이스를 하나의 L2 네트워크로 연결하고, Ethernet Frame을 전달할 수 있다.

VM의 가상 인터페이스와 물리 인터페이스를 연결하는 데 사용할 수 있다.

Open vSwitch(OVS)

Open vSwitch는 가상화 환경을 위한 소프트웨어 기반 가상 스위치이다.

일반적인 L2 Switching 기능 외에도 VLAN, 터널링, 프로그래밍 가능한 패킷 처리 기능 등을 지원한다.

주요 특징은 다음과 같다.

- 가상 네트워크 인터페이스 연결
- VLAN을 이용한 네트워크 구분
- GRE 및 VXLAN 기반 Overlay Network 지원
- 외부 네트워크 장비와 연결
- 네트워크 트래픽 처리 규칙 관리

VLAN

VLAN(Virtual Local Area Network)은 하나의 물리 네트워크를 여러 논리적인 L2 네트워크로 구분하는 기술이다.

예를 들어 하나의 물리 스위치에서 VLAN 10과 VLAN 20을 구성할 수 있다.

```
Physical Switch
     |
     +--- VLAN 10
     |       |
     |      VM1
     |
     +--- VLAN 20
             |
            VM2
```

VM1과 VM2는 동일한 물리 스위치를 사용하더라도 서로 다른 L2 네트워크에 속한다.

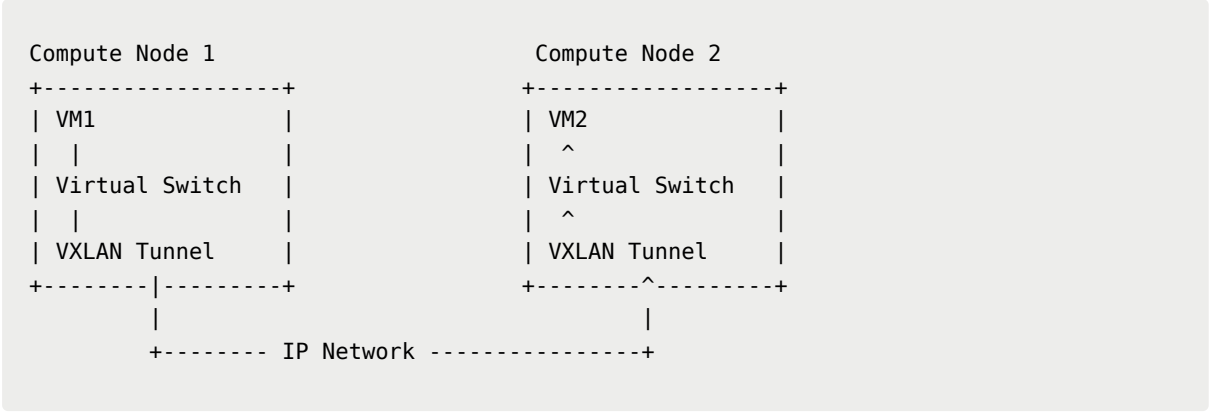
따라서 서로 다른 VLAN 간 통신에는 적절한 L3 Routing이 필요하다.

GRE와 VXLAN

GRE와 VXLAN은 기존 네트워크 위에서 다른 네트워크의 패킷을 캡슐화하여 전달하는 터널링 기술이다.

이를 통해 물리적으로 서로 다른 서버에 위치한 VM들을 하나의 논리적인 네트워크에 연결할 수 있다.

예를 들어 서로 다른 Compute Node에 있는 VM1과 VM2가 VXLAN 기반 가상 네트워크를 사용하는 경우 다음과 같은 흐름이 발생한다.



1. VM1에서 Ethernet Frame이 발생한다.
2. 가상 스위치가 프레임을 처리한다.
3. 기존 프레임이 VXLAN 패킷으로 캡슐화된다.
4. 기반 IP 네트워크를 통해 다른 Compute Node로 전달된다.
5. 목적지에서 VXLAN 캡슐화를 해제한다.
6. 원래 Ethernet Frame이 VM2로 전달된다.

이러한 방식을 이용하면 물리 네트워크의 구조와 독립적인 논리적 네트워크를 구성할 수 있다.

Switching의 핵심은 동일한 L2 네트워크에서 프레임을 전달하는 것이며, Linux Bridge와 OVS가 이를 구현하고 VLAN 및 터널링 기술이 네트워크 분리와 확장을 지원한다는 점이다.

2.2 Routing

Routing은 **Layer 3(Network Layer)** 에서 서로 다른 IP 네트워크 사이에 패킷을 전달하는 기능이다.

Switching이 같은 L2 네트워크 내에서 프레임을 전달한다면, Routing은 서로 다른 네트워크를 연결한다.

예를 들어 다음 두 네트워크가 있다고 가정한다.

네트워크	IP 대역
Network A	192.168.10.0/24

네트워크	IP 대역
Network B	192.168.20.0/24

Network A의 VM1이 Network B의 VM2와 통신하기 위해서는 L3 Routing이 필요하다.

```
VM1
192.168.10.10
    |
Network A
    |
 Router
    |
Network B
    |
VM2
192.168.20.10
```

라우터는 목적지 IP 주소와 Routing Table을 이용하여 패킷을 전달할 경로를 결정한다.

Neutron Router

Neutron은 소프트웨어 기반의 가상 라우터를 구성할 수 있다.

Neutron Router는 다음과 같은 기능을 제공할 수 있다.

- 서로 다른 Tenant Network 연결
- 내부 네트워크와 외부 네트워크 연결
- IP Forwarding
- NAT
- Floating IP를 이용한 외부 접근 지원

이러한 기능은 Linux의 네트워크 기능을 활용하여 구현된다.

IP Forwarding

IP Forwarding은 한 네트워크 인터페이스에서 받은 IP 패킷을 다른 인터페이스로 전달하는 기능이다.

라우터는 Routing Table을 참고하여 패킷의 목적지에 맞는 전달 경로를 선택한다.

Network Namespace

Linux Network Namespace는 독립적인 네트워크 환경을 제공하는 기능이다.

각 Namespace는 독립적인 네트워크 인터페이스, IP 주소, Routing Table, 방화벽 규칙 등을 가질 수 있다.

이를 통해 하나의 Linux 호스트에서 여러 가상 라우터를 서로 격리하여 실행할 수 있다.

특히 여러 Tenant가 동일한 사설 IP 대역을 사용하는 환경에서 독립적인 네트워크 공간을 제공하는 데 활용된다.

NAT(Network Address Translation)

NAT는 네트워크 패킷의 IP 주소를 변환하는 기능이다.

대표적인 방식은 다음과 같다.

NAT 종류	역할
SNAT	출발지 IP 주소 변경
DNAT	목적지 IP 주소 변경

예를 들어 사설 IP를 사용하는 VM이 외부 네트워크에 접근할 때 SNAT를 사용하여 출발지 IP를 외부 통신용 주소로 변환할 수 있다.

반대로 외부 클라이언트가 Floating IP로 VM에 접근할 때 DNAT가 사용될 수 있다.

Routing은 서로 다른 네트워크를 연결하는 기능이고, NAT는 필요한 경우 주소를 변환하는 추가 기능이다.

2.3 Switching과 Routing 비교

구분	Switching	Routing
네트워크 계층	L2	L3
주요 판단 정보	MAC 주소	IP 주소
역할	동일 L2 네트워크 내 전달	서로 다른 IP 네트워크 간 전달
대표 구현	Linux Bridge, OVS	Linux IP Routing
관련 기술	VLAN, GRE, VXLAN	IP Forwarding, NAT

실제 VM 네트워크 경로에서는 Switching과 Routing이 함께 사용될 수 있다.

예를 들어 VM이 외부 네트워크에 접근하는 경우 가상 스위치가 패킷을 라우터까지 전달하고, 라우터가 목적지 네트워크를 결정하여 다음 네트워크로 전달한다.

2.4 Load Balancing

Load Balancing은 하나의 서비스로 들어오는 요청을 여러 서버나 VM으로 분산하는 기능이다.

하나의 서비스 주소로 들어오는 요청을 여러 백엔드 VM에 분산하여 특정 서버에 부하가 집중되는 것을 줄일 수 있다.

예를 들어 다음과 같이 구성할 수 있다.

```
Client
   |
Load Balancer
   |
   +--- VM1
   |
   +--- VM2
   |
   +--- VM3
```

2.5 Firewalling

Firewalling은 네트워크 트래픽을 정해진 보안 정책에 따라 허용하거나 차단하는 기능이다.

Neutron에서는 대표적으로 다음 두 기능을 구분할 수 있다.

Security Group

VM 네트워크 인터페이스에 적용되는 보안 정책이다.

예를 들어 SSH 접근을 허용하거나 특정 네트워크 대역에서의 접속을 차단할 수 있다.

초기 Neutron의 Security Group은 Compute Node에서 iptables를 이용해 트래픽을 필터링하는 방식으로 구현되었다.

Firewall-as-a-Service(FWaaS)

가상 라우터 등의 네트워크 지점에서 방화벽 정책을 제공하기 위한 확장 기능이다.

Security Group이 인스턴스 네트워크 트래픽의 보호에 사용되는 것과 달리 FWaaS는 라우터를 통한 네트워크 트래픽에 정책을 적용하는 구조를 제공한다.

2.6 Virtual Private Networks

VPN은 공용 네트워크를 이용하여 떨어져 있는 사설 네트워크를 논리적으로 연결하는 기술이다.

예를 들어 클라우드 내부 네트워크와 원격 네트워크를 암호화된 터널로 연결할 수 있다.

Neutron은 IPSec 기반 VPN 연결을 관리하기 위한 API 확장 기능인 VPNaaS를 제공하도록 설계되었다.

---

3. Preparing the Physical Infrastructure

OpenStack Networking은 소프트웨어 기반의 가상 네트워크를 제공하지만, 실제 네트워크 트래픽은 물리 인프라를 통해 전달된다.

서로 다른 Compute Node에 존재하는 VM들이 통신하려면 서버의 물리 NIC와 스위치 등을 통한 연결이 필요하다.

따라서 OpenStack을 구축하기 전에 물리 네트워크가 어떤 트래픽을 처리해야 하는지 결정해야 한다.

3.1 물리 인프라 설계 시 고려사항

네트워크 설계에서는 다음 요소를 고려한다.

- 클라우드의 목적과 사용 규모
- 물리 서버 및 NIC 수
- 네트워크 성능 요구사항
- 보안 및 트래픽 격리 요구사항
- 네트워크 장애에 대한 복원력
- 외부 네트워크와의 연결 방식

예를 들어 개발과 테스트가 목적이라면 단순한 네트워크 구조를 사용할 수 있다.

반면 운영 환경에서는 대역폭 경합을 줄이고 장애 영향을 제한하기 위해 여러 NIC와 네트워크 분리 방식을 고려할 수 있다.

3.2 물리 네트워크와 가상 네트워크

OpenStack 네트워크는 물리 네트워크와 가상 네트워크가 결합된 구조이다.

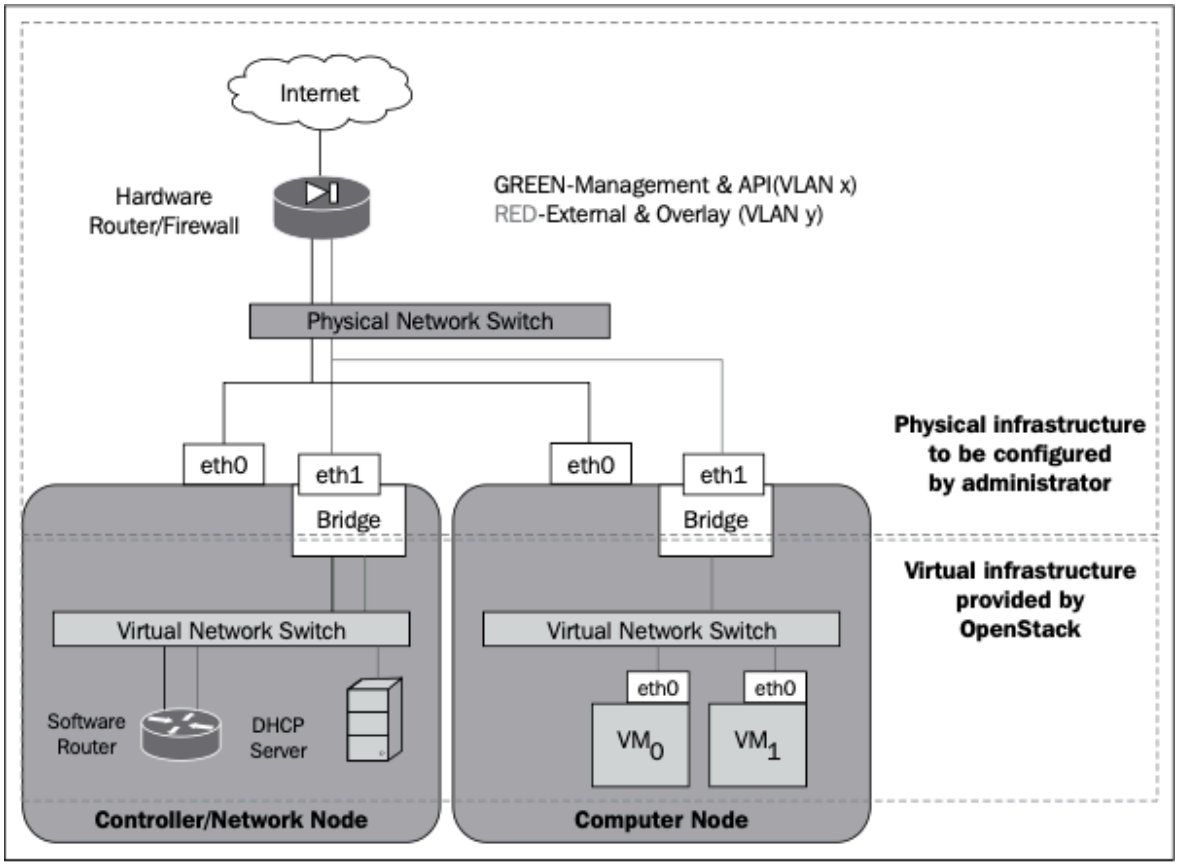
물리 네트워크 구성요소

- Physical NIC
- Physical Switch
- Physical Router / Firewall
- 서버 간 연결

가상 네트워크 구성요소

- Virtual NIC
- Virtual Switch
- Virtual Router

- Tenant Network
- Overlay Network



가상 네트워크가 정상적으로 설정되어 있더라도 물리 연결에 문제가 발생하면 VM 통신이 실패할 수 있다.

따라서 가상 네트워크의 동작은 이를 뒷받침하는 물리 네트워크의 연결성에도 의존한다.

---

4. Types of Network Traffic

OpenStack 환경에서는 서로 다른 목적을 가진 네트워크 트래픽이 발생한다.

대표적인 유형은 다음 네 가지이다.

1. Management Network
2. API Network
3. External Network
4. Guest Network

각 네트워크의 목적은 다르지만, 반드시 서로 다른 물리 NIC를 사용해야 하는 것은 아니다.

4.1 Management Network

Management Network는 OpenStack 내부 서비스와 서버 사이의 통신을 위한 네트워크이다.

주로 다음과 같은 통신에 사용된다.

- Database Service
- Messaging Service
- 내부 OpenStack 서비스 간 통신
- 서버 관리 통신

예를 들어 서비스가 데이터베이스 또는 메시징 서비스와 통신할 때 Management Network를 사용할 수 있다.

이 네트워크에 장애가 발생하면 OpenStack 내부 서비스 간 통신에 영향을 줄 수 있다.

4.2 API Network

API Network는 사용자와 다른 서비스가 OpenStack API에 접근하기 위해 사용하는 네트워크이다.

대표적인 API 서비스는 다음과 같다.

- Keystone
- Nova
- Neutron
- Glance

Horizon과 같은 관리 인터페이스를 제공하는 서비스에 접근하는 경로에도 활용할 수 있다.

Management Network와 API Network는 논리적으로 다른 역할을 가지지만, 동일한 물리 인터페이스를 사용하도록 구성할 수 있다.

4.3 External Network

External Network는 Neutron Router가 외부 네트워크와 연결되는 경로를 제공한다.

주요 역할은 다음과 같다.

- 내부 VM의 외부 네트워크 접근
- Neutron Router의 외부 게이트웨이 연결
- Floating IP 주소 제공

- 외부 클라이언트와 VM 사이의 통신 경로 제공

예를 들어 내부 VM이 외부 웹 서버에 접근한다면 다음과 같은 흐름이 사용될 수 있다.

```
VM
 |
Guest Network
 |
Neutron Router
 |
External Network
 |
External Router
 |
Internet
```

Floating IP를 이용하여 외부에서 VM으로 접근하는 경우에도 External Network가 사용된다.

4.4 Guest Network

Guest Network는 VM에서 발생하는 실제 사용자 데이터 트래픽을 처리하는 네트워크이다.

대표적인 예시는 다음과 같다.

- VM1과 VM2 사이의 통신
- 서로 다른 Compute Node에 위치한 VM 간 통신
- VM에서 네트워크 게이트웨이로 전달되는 트래픽

Guest Network를 구성하는 방식은 다음과 같다.

방식	설명
Local	특정 노드 내부에서 사용되는 네트워크
Flat	VLAN 태그 없이 물리 네트워크에 연결
VLAN	VLAN ID를 이용한 논리적 네트워크 분리
GRE	GRE 터널 기반 Overlay Network
VXLAN	VXLAN 터널 기반 Overlay Network

Guest Network는 물리 NIC를 독립적으로 사용하거나 다른 네트워크 트래픽과 공유하도록 구성할 수 있다.

4.5 네 가지 네트워크 비교

Network	주요 목적	트래픽 예시
Management	내부 서비스 통신	Database, Messaging
API	OpenStack API 접근	Neutron API 요청
External	외부 네트워크 연결	Floating IP, External Gateway
Guest	VM의 데이터 통신	VM 간 통신, Overlay

5. Physical Server Connections

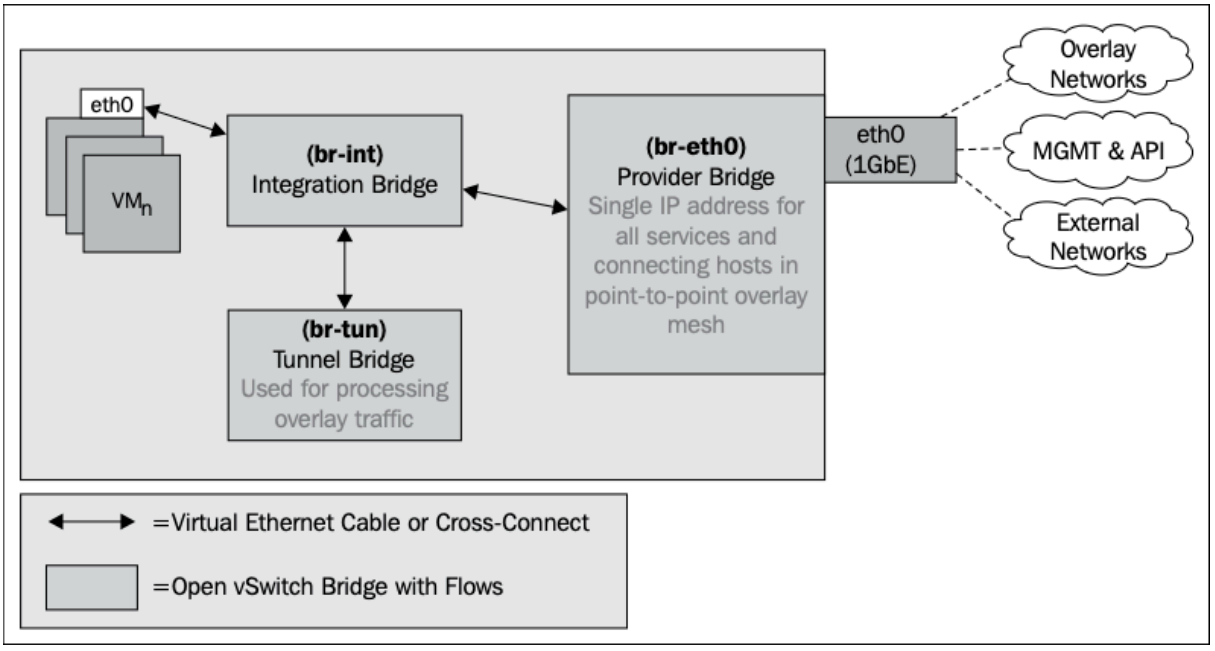
OpenStack 서버에서는 물리 NIC를 이용하여 관리 통신, API 접근, VM 트래픽 등을 전달한다.

물리 인터페이스의 수와 연결 방식에 따라 네트워크 성능 및 장애 영향 범위가 달라진다.

대표적인 연결 방식은 Single Interface, Multiple Interfaces, Bonding이다.

5.1 Single Interface

Single Interface는 하나의 물리 NIC를 통해 모든 유형의 트래픽을 전송하는 방식이다.



하나의 NIC에서 다음 트래픽이 함께 처리된다.

- Management
- API
- External
- Guest / Overlay

장점

- 추가 NIC가 필요하지 않다.
- 네트워크 연결 구성이 단순하다.
- 소규모 테스트 및 PoC 환경에 적합하다.

단점

- 여러 네트워크 트래픽이 동일한 대역폭을 공유한다.
- VM에서 많은 트래픽이 발생하면 관리 통신에도 영향을 줄 수 있다.
- NIC 장애 발생 시 여러 네트워크 기능이 함께 영향을 받는다.
- 트래픽을 물리적으로 분리하기 어렵다.

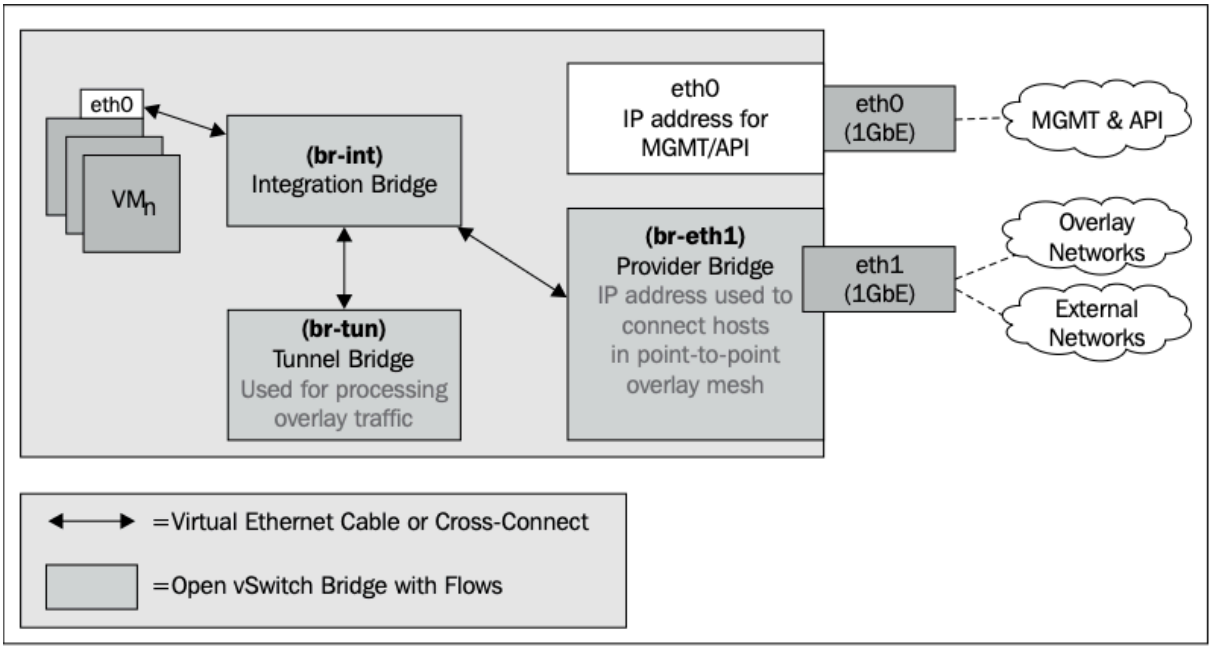
예를 들어 하나의 1Gbps NIC를 통해 모든 트래픽을 처리한다면, Guest Network에서 대량의 데이터를 전송할 때 Management 및 API 트래픽이 사용할 수 있는 대역폭이 줄어들 수 있다.

따라서 단순한 환경에서는 사용할 수 있지만, 높은 안정성과 성능이 필요한 운영 환경에서는 주의가 필요하다.

5.2 Multiple Interfaces

Multiple Interfaces는 두 개 이상의 물리 NIC를 이용하여 서로 다른 목적의 트래픽을 분리하는 방식이다.

예를 들어 다음과 같이 구성할 수 있다.



eth0

- Management Traffic
- API Traffic

eth1

- External Traffic
- Guest Traffic
- Overlay Network

이러한 분리를 통해 Guest Network의 대역폭 사용이 Management Network에 미치는 영향을 줄일 수 있다.

장점

- 서로 다른 트래픽의 대역폭 경합 감소
- 관리 네트워크와 VM 네트워크 분리
- 네트워크별 보안 정책 적용에 유리
- 네트워크 장애 영향 범위를 구분하기 쉬움

단점

- 추가 NIC와 스위치 포트가 필요하다.

- 물리 네트워크 구성이 복잡해진다.
- 각 인터페이스와 연결된 네트워크 설정을 정확하게 관리해야 한다.

Multiple Interfaces는 네트워크 성능과 보안 분리가 중요한 환경에서 사용하기 적합하다.

5.3 Bonding

Bonding은 두 개 이상의 물리 NIC를 하나의 논리적인 네트워크 인터페이스로 결합하는 기술이다.

주요 목적은 다음과 같다.

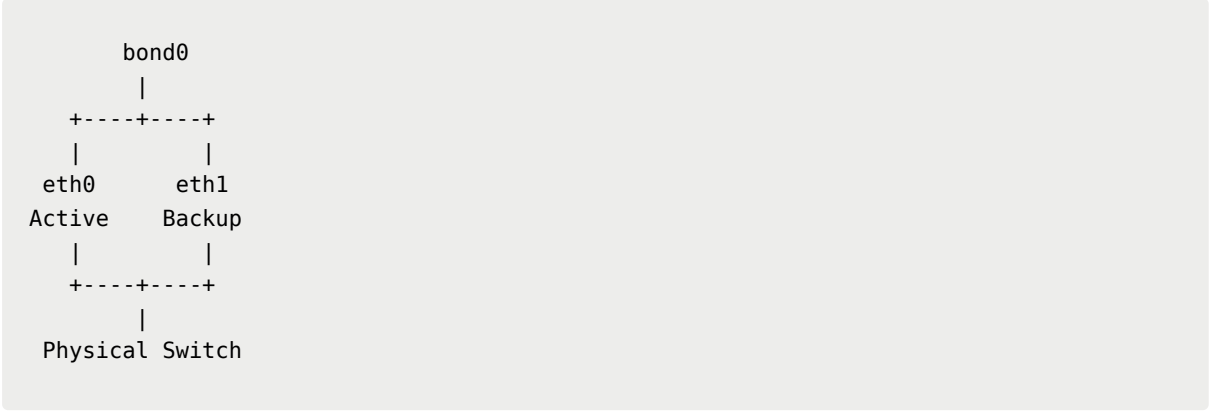
대역폭 확장

여러 물리 링크를 결합하여 트래픽을 분산함으로써 전체 네트워크 처리량을 높일 수 있다. 단, 실제 성능은 Bonding 모드와 트래픽 분산 방식에 따라 달라진다.

네트워크 이중화

하나의 물리 링크에 장애가 발생했을 때 다른 링크를 사용하여 통신을 유지할 수 있다.

대표적인 방식은 Active/Backup이다.



Active 링크에 장애가 발생하면 Backup 링크가 통신을 이어받는다. 이러한 구성을 이용하면 단일 NIC 장애에 대한 복원력을 높일 수 있다.

5.4 구성 방식 비교

항목	Single Interface	Multiple Interfaces	Bonding
목적	단순한 연결	트래픽 분리	대역폭 확장 및 이중화
NIC 수	1개	2개 이상	일반적으로 2개 이상
트래픽 분리	제한적	가능	별도 네트워크 설계에 따라 가능
링크 장애 대응	취약	구성에 따라 다름	이중화 모드에서 대응
구성 복잡도	낮음	중간	상대적으로 높음

Multiple Interfaces와 Bonding은 서로 대체하는 기술이 아니다.

여러 인터페이스로 트래픽을 분리하면서 각각의 네트워크 연결에 Bonding을 적용할 수도 있다.

---

6. Separating Services Across Nodes

OpenStack Networking 서비스는 하나의 노드에 집중하여 배치하거나 여러 노드에 분산하여 실행할 수 있다.

대표적인 노드 역할은 다음과 같다.

노드	주요 역할
Controller Node	API, Database, Messaging, OpenStack 관리 서비스
Network Node	가상 라우팅, DHCP, Metadata, 네트워크 서비스
Compute Node	VM 실행 및 VM 네트워크 연결

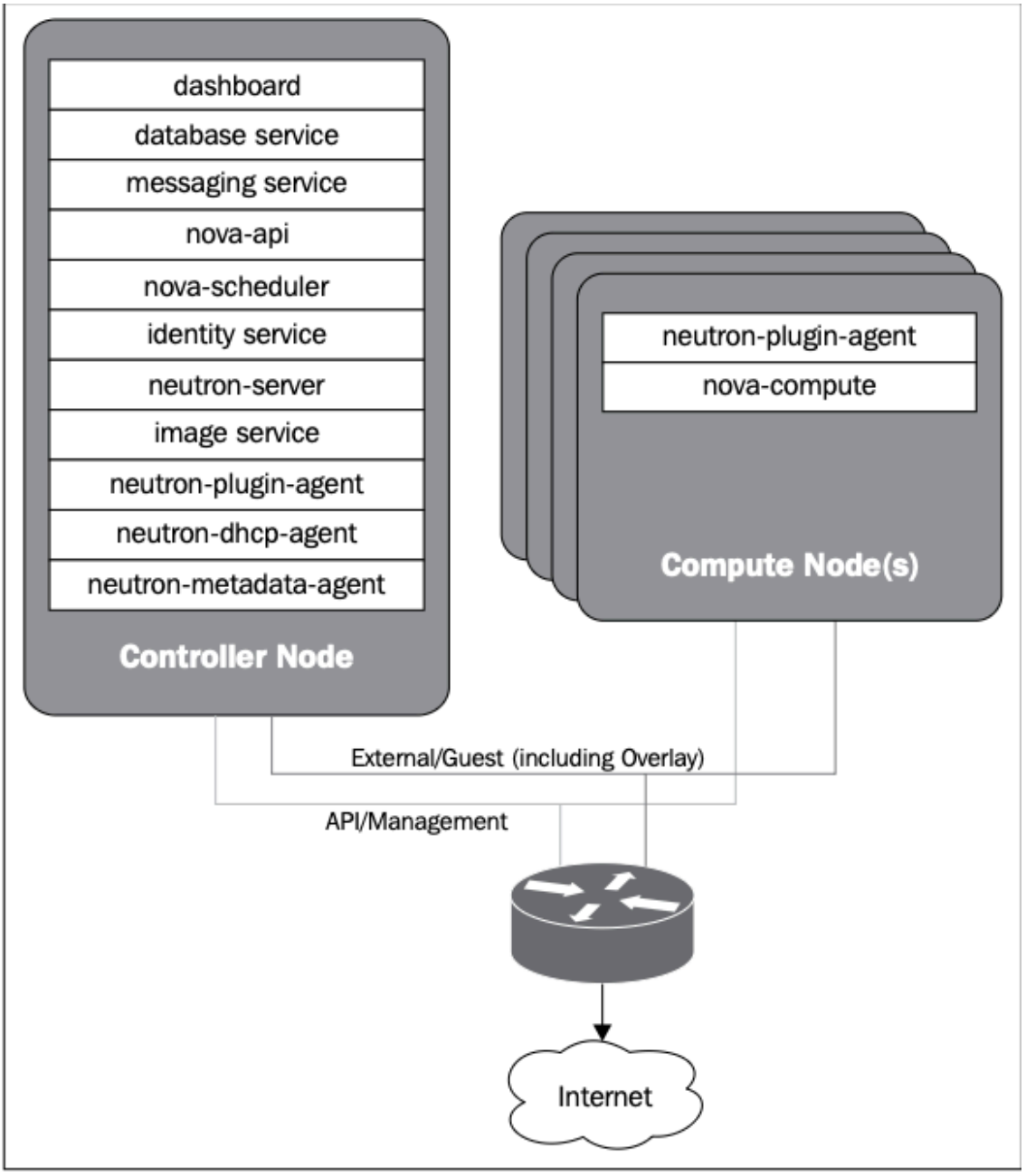
서비스 배치 방식에 따라 네트워크 트래픽의 처리 위치와 장애 영향 범위가 달라진다.

6.1 Controller + Compute Nodes — L2 Only

이 구성에서는 Controller Node가 대부분의 OpenStack 관리 서비스와 Neutron 네트워크 서비스를 실행한다.

Compute Node는 VM 실행과 가상 스위칭을 담당한다.

Neutron L3 Agent를 사용하지 않는 경우, Neutron은 VM을 위한 L2 연결을 제공하고 서로 다른 네트워크 사이의 라우팅은 외부 라우터가 담당한다.

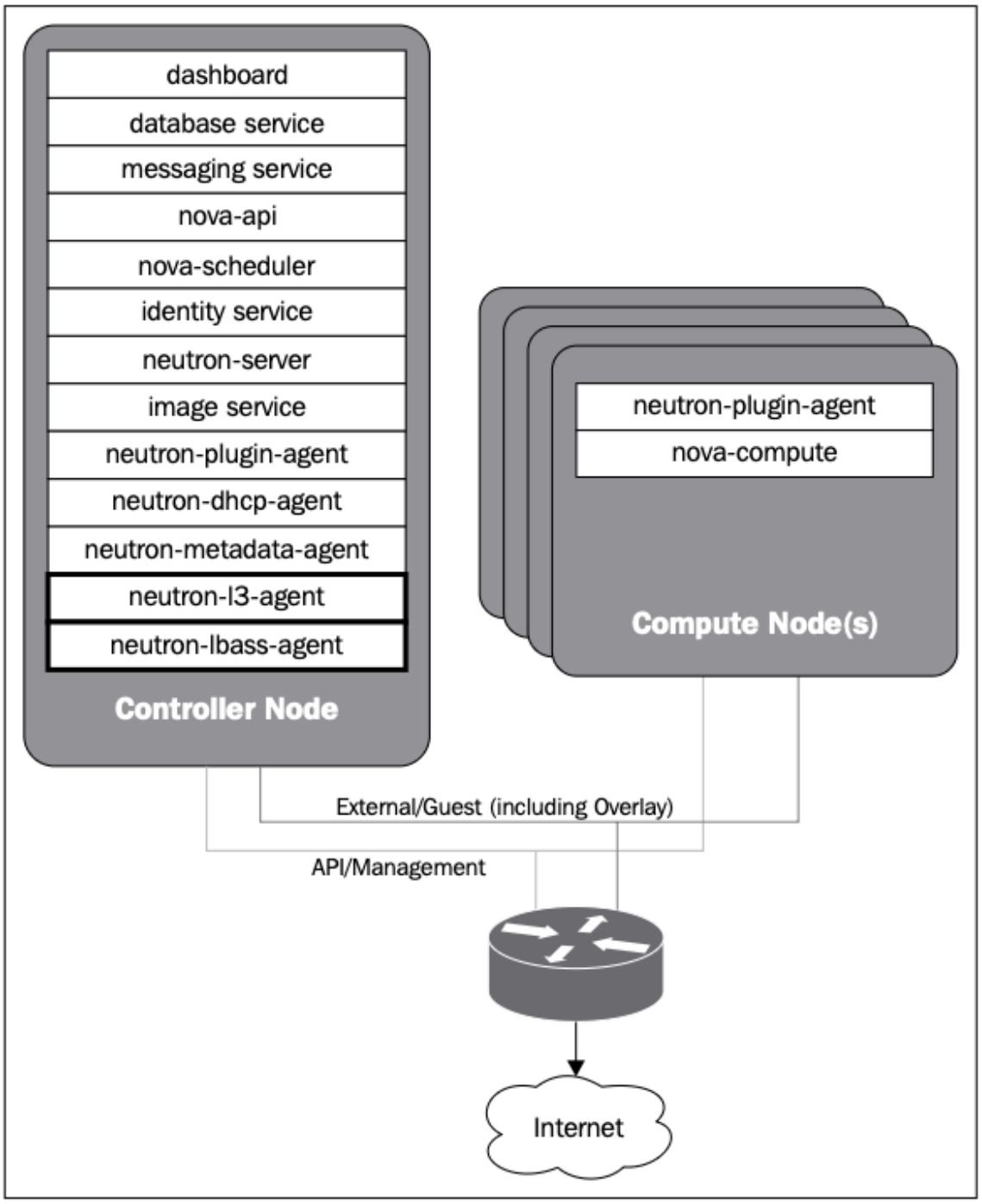


특징

- 구성 구조가 비교적 단순하다.
- Controller에 관리 및 네트워크 서비스가 집중된다.
- Neutron의 L2 연결 기능을 이용한다.
- 네트워크 간 라우팅은 외부 라우터에 의존한다.

6.2 Controller + Compute Nodes — L3 Agent 포함

Controller Node에서 Neutron L3 Agent를 실행하면 가상 라우터를 통해 서로 다른 네트워크를 연결할 수 있다.



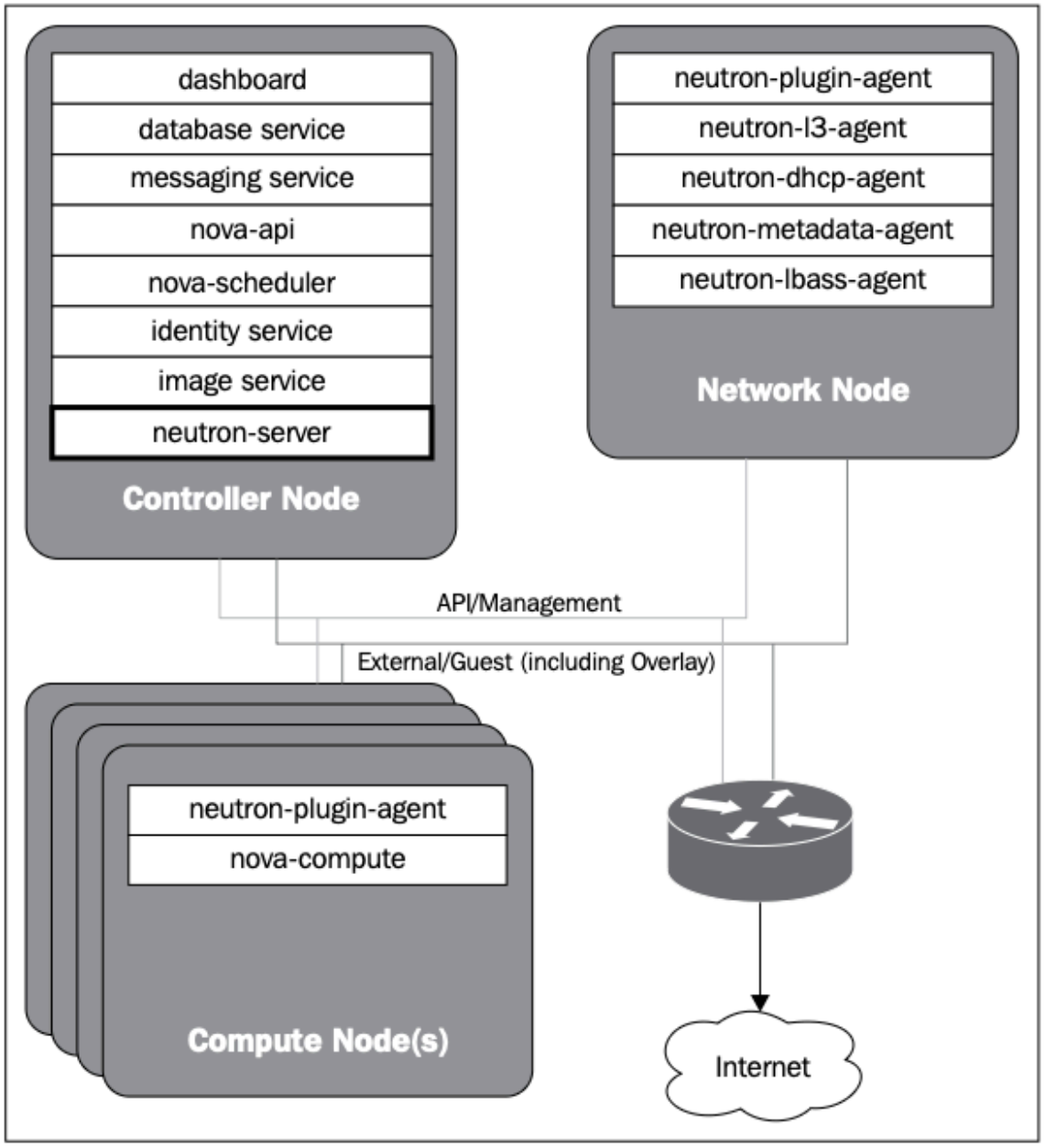
이 구성에서는 Controller Node가 OpenStack 관리 서비스뿐 아니라 L3 Routing도 담당한다.

특징

- Neutron Router를 이용한 네트워크 간 통신 지원
- 별도의 Network Node 없이 구성 가능
- Controller에 관리 및 네트워크 처리 기능 집중
- Controller 장애 또는 자원 부족 시 여러 서비스가 함께 영향을 받을 수 있음

6.3 Controller + Network + Compute Nodes

전용 Network Node를 구성하여 네트워크 서비스를 분리할 수 있다.



Network Node는 라우팅, DHCP, Metadata 등의 네트워크 서비스를 담당한다.

Controller Node는 Neutron API 및 관리 기능을 담당한다.

이를 통해 Controller에서 발생하는 관리 작업과 Network Node에서 처리하는 네트워크 작업을 분리할 수 있다.

장점

- Controller의 자원 경합 감소

- 네트워크 처리 기능을 별도 노드에서 관리
- 서비스 역할에 따른 자원 배치 가능
- 네트워크 기능의 확장 및 이중화 설계에 유리

다만 모든 네트워크 서비스가 단일 Network Node에 집중되어 있다면 해당 노드의 장애가 여러 네트워크에 영향을 미칠 수 있다.

6.4 노드 구성 비교

구성	L3 처리 위치	특징
Controller + Compute (L2 Only)	외부 라우터	L2 연결 중심
Controller + Compute (L3)	Controller	관리와 라우팅 기능 집중
Controller + Network + Compute	Network Node	네트워크 서비스 분리

노드 구성 방식은 클라우드의 규모, 트래픽, 보안 요구사항 및 가용성 요구사항에 따라 결정된다.

Chapter 3. Installing Neutron

7. Basic Neutron Constructs

Neutron은 API를 통해 가상 네트워크를 구성하고 관리한다.

Neutron 네트워크를 이해하기 위한 가장 기본적인 리소스는 다음 세 가지이다.

- Network
- Subnet
- Port

각각의 리소스는 독립적인 의미를 가지지만 서로 연결되어 하나의 가상 네트워크를 구성한다.

7.1 Network

Network는 독립된 **Layer 2 Broadcast Domain** 을 나타내는 논리적인 네트워크 리소스이다.

Network는 VM 및 다른 네트워크 장치들이 연결되는 논리적인 L2 영역을 제공한다.

기본적으로 Tenant가 자신이 생성한 Network를 사용하지만, 설정에 따라 다른 Tenant와 공유할 수도 있다.

Network는 Neutron API의 핵심 리소스이며, Subnet과 Port는 반드시 특정 Network와 연결되어야 한다.

Network의 역할

- 논리적인 L2 네트워크 생성
- VM 간 네트워크 연결 영역 제공
- Tenant Network 격리
- Subnet 및 Port를 연결하는 기반 리소스 제공

예를 들어 Tenant A가 `network-a`라는 가상 네트워크를 생성하면, 해당 네트워크에 Subnet과 Port를 추가하여 VM을 연결할 수 있다.

7.2 Subnet

Subnet은 Network에 연결되는 IPv4 또는 IPv6 주소 범위이다.

Subnet에는 IP 주소의 범위를 나타내는 CIDR 정보가 필요하다.

예를 들어 다음과 같은 Subnet을 생성할 수 있다.

```
Network: network-a

Subnet:
192.168.10.0/24
```

Subnet은 VM에 할당할 수 있는 IP 주소 범위를 정의한다.

한 Network에 여러 Subnet을 연결할 수도 있으며, Subnet들은 반드시 연속된 주소 범위를 가질 필요는 없다.

예를 들어 하나의 Network에 다음 두 Subnet을 연결할 수 있다.

```
Network: network-a
    |
    +--- Subnet A: 192.168.10.0/24
    |
    +--- Subnet B: 192.168.20.0/24
```

또한 DHCP Allocation Pool을 설정하면 VM에 자동으로 할당할 주소 범위를 제한할 수 있다.

Network와 Subnet의 차이

Network는 논리적인 L2 연결 영역을 나타내고, Subnet은 해당 Network에서 사용되는 IP 주소 범위를 나타낸다. 따라서 Network와 Subnet은 같은 개념이 아니다.

7.3 Port

Port는 Neutron이 관리하는 가상 스위치의 논리적인 포트이다. VM의 가상 네트워크 인터페이스는 Neutron Port에 매핑된다.

Port는 해당 인터페이스에 할당되는 네트워크 정보를 정의한다.

대표적인 정보는 다음과 같다.

- MAC Address
- IP Address
- 연결된 Network
- 가상 인터페이스 연결 정보

예를 들어 다음과 같은 Port가 있다고 가정한다.

```
Port A
- Network: network-a
- MAC: fa:16:3e:aa:bb:cc
- IP: 192.168.10.10
```

VM이 Port A에 연결되면 해당 Port에 정의된 네트워크 정보를 이용하여 네트워크에 연결된다.

Port 정보는 Neutron Database에 저장되며, 선택한 Network Plugin의 Agent가 가상 네트워크 연결을 구성하는 데 사용한다.

7.4 Network, Subnet, Port의 관계

세 리소스의 관계를 정리하면 다음과 같다.

```
Network: network-a
     |
     +--- Subnet
     |    192.168.10.0/24
     |
     +--- Port A
     |    MAC: fa:16:3e:aa:bb:cc
     |    IP : 192.168.10.10
     |          |
     |         VM1
     |
     +--- Port B
```

```
          MAC: fa:16:3e:dd:ee:ff
          IP : 192.168.10.20
               |
              VM2
```

각 리소스의 역할은 다음과 같다.

리소스	의미	주요 정보
Network	L2 네트워크	네트워크 식별 정보
Subnet	IP 주소 범위	CIDR, DHCP 할당 범위
Port	가상 네트워크 연결 지점	MAC, IP, Network 연결 정보

이 구조를 통해 Nova는 VM의 가상 네트워크 인터페이스를 Neutron Port에 연결할 수 있다.

따라서 **VM은 Port를 통해 Network에 연결되며, Subnet은 해당 네트워크에서 사용할 IP 주소 범위를 정의한다.**

---

8. Overlapping Networks Using Network Namespaces

OpenStack은 여러 Tenant가 같은 클라우드 인프라를 공유하는 멀티테넌트 환경을 지원한다.

멀티테넌트 환경에서는 각 Tenant가 자신만의 네트워크, 라우터 및 관련 네트워크 리소스를 구성할 수 있어야 한다.

이 과정에서 서로 다른 Tenant가 동일한 사설 IP 대역을 사용할 수 있다.

예를 들어 다음과 같은 상황을 가정한다.

```
Tenant A
Network: 192.168.10.0/24

Tenant B
Network: 192.168.10.0/24
```

두 Tenant는 동일한 IP 주소 범위를 사용하지만 서로 독립된 네트워크로 동작해야 한다.

이러한 네트워크 격리를 제공하기 위한 Linux 기능 중 하나가 **Network Namespace**이다.

8.1 Network Namespace란?

Network Namespace는 하나의 Linux 시스템 안에서 독립적인 네트워크 환경을 생성하는 기능이다.

각 Namespace는 서로 다른 네트워크 구성을 가질 수 있다.

주요 격리 대상은 다음과 같다.

- Network Interface
- IP Address
- Routing Table
- Firewall Rules
- 네트워크 관련 소켓 및 설정

예를 들어 Namespace A와 Namespace B가 각각 독립적인 Routing Table을 가진다면, 동일한 목적지 IP에 대해서도 서로 다른 라우팅 경로를 구성할 수 있다.

이를 이용하여 서로 다른 Tenant의 네트워크 서비스를 하나의 물리 호스트에서 격리하여 실행할 수 있다.

8.2 Neutron의 Network Namespace

초기 Neutron 구현에서는 DHCP, Router, Load Balancer 등의 네트워크 서비스를 Namespace를 통해 분리할 수 있다.

대표적인 Namespace는 다음과 같다.

Namespace	역할
`qdhcp-<network UUID>`	특정 Network의 DHCP 서비스
`qrouter-<router UUID>`	특정 Router의 라우팅 서비스
`qlbaas-<load balancer UUID>`	특정 Load Balancer의 트래픽 처리

qdhcp Namespace

`qdhcp` Namespace는 Network에 연결된 VM에 DHCP 서비스를 제공한다.

해당 Network에 연결된 인터페이스를 통해 DHCP 요청을 처리할 수 있다.

주요 역할은 다음과 같다.

- DHCP 요청 수신
- IP 주소 및 네트워크 설정 제공
- Network 단위 DHCP 서비스 격리

qrouter Namespace

`qrouter` Namespace는 Neutron Router의 네트워크 환경을 제공한다.

Router가 연결된 여러 네트워크 사이에서 패킷을 전달한다.

대표적인 기능은 다음과 같다.

- Routing Table 관리
- IP Forwarding
- NAT
- 내부 네트워크와 외부 네트워크 연결

qlbaas Namespace

`qlbaas` Namespace는 초기 Neutron LBaaS에서 Load Balancer를 격리하여 실행하는 데 사용되었다.

HAProxy 등의 로드밸런서 프로세스가 실행될 수 있으며, 해당 네트워크의 VM으로 요청을 분산할 수 있다.

8.3 Namespace를 이용한 네트워크 격리

예를 들어 서로 다른 Tenant가 동일한 IP 대역을 사용하는 경우에도 독립적인 Router Namespace를 이용하여 서로 다른 라우팅 환경을 유지할 수 있다.

```
Linux Host
|
+--- Tenant A
|    |
|    +--- Router Namespace A
|         |
|         +--- Routing Table A
|         +--- 192.168.10.0/24
|
+--- Tenant B
     |
     +--- Router Namespace B
          |
          +--- Routing Table B
          +--- 192.168.10.0/24
```

두 Namespace가 같은 IP 주소 범위를 사용하더라도 네트워크 인터페이스와 라우팅 환경이 분리되어 있기 때문에 서로 독립적인 네트워크를 구성할 수 있다.

Namespace는 주로 Controller Node 또는 전용 Network Node에서 사용된다.

Network Namespace의 핵심은 하나의 Linux 호스트 안에서 독립적인 네트워크 환경을 구성하여 여러 네트워크 서비스를 격리할 수 있다는 점이다.

---

9. Extending Network Functions with Plugins

Neutron은 다양한 네트워크 기술을 지원하기 위해 Plugin 구조를 제공한다.

Neutron 이전의 `nova-network`는 Linux Bridge, VLAN, iptables 등을 활용하여 기본적인 네트워크 기능을 제공했다.

Neutron은 Plugin 구조를 통해 여러 네트워크 기술과 하드웨어·소프트웨어 구현을 연동할 수 있도록 한다.

9.1 Network Plugin의 역할

Network Plugin은 Neutron API로 요청된 네트워크 구성을 실제 네트워크 기술과 연결하는 역할을 수행한다.

예를 들어 사용자가 Network를 생성하면, 선택된 Plugin이 해당 Network를 구현하기 위한 처리를 담당한다.

```
User / Nova
     |
Neutron API
     |
neutron-server
     |
Network Plugin
     |
Network Agent
     |
Virtual Network
```

사용자가 사용하는 Neutron API는 일관된 인터페이스를 제공하며, 내부 네트워크 구현 기술은 Plugin에 따라 달라질 수 있다.

9.2 LinuxBridge Plugin

LinuxBridge Plugin은 Linux Bridge를 사용하여 인스턴스의 L2 연결을 구현한다.

VLAN이나 Overlay 기술을 이용하여 가상 네트워크를 구성할 수 있다.

9.3 Open vSwitch Plugin

Open vSwitch Plugin은 OVS를 사용하여 가상 네트워크의 L2 연결을 구현한다.

OVS를 통해 가상 인터페이스 연결, VLAN 처리, 터널 기반 네트워크 등을 구성할 수 있다.

9.4 Monolithic Plugin과 ML2

초기 LinuxBridge Plugin과 Open vSwitch Plugin은 Monolithic Plugin 구조로 제공되었다.

Monolithic Plugin은 각 플러그인이 독립적으로 네트워크 구현을 담당하는 방식으로, 이러한 플러그인들을 동시에 사용하는 데 제약이 있었다.

이를 개선하기 위해 **Modular Layer 2(ML2)** Plugin이 등장했다.

ML2는 여러 L2 네트워크 기술을 보다 유연하게 지원하기 위한 구조이다.

구분	초기 Monolithic Plugin	ML2
구조	개별 네트워크 플러그인 중심	모듈화된 L2 플러그인 구조
대표 기술	LinuxBridge, OVS	여러 L2 네트워크 기술
특징	플러그인 조합에 제약	다양한 네트워크 기술 연동 지원

LinuxBridge와 Open vSwitch의 초기 Monolithic Plugin은 Icehouse에서 ML2로 대체되는 방향으로 변경되었다.

ML2의 상세 구성은 이 장의 범위에 포함되지 않으며, Appendix B에서 별도로 다룬다.

---

10. Configuring Neutron Services

Neutron은 하나의 서비스만으로 모든 네트워크 기능을 처리하지 않는다.

API 요청을 처리하는 `neutron-server`와 DHCP, Metadata, Routing 등 각 네트워크 기능을 담당하는 Agent가 함께 동작한다.

각 구성요소가 담당하는 기능을 구분하면 네트워크 서비스 간의 관계를 이해할 수 있다.

10.1 neutron-server

`neutron-server`는 Neutron API를 제공하고, 사용자의 API 요청을 적절한 Network Plugin에 전달하는 서비스이다.

주요 역할은 다음과 같다.

- Neutron API 요청 수신
- Network, Subnet, Port 등의 리소스 관리
- Network Plugin을 통한 네트워크 구성 처리
- Database와 연동하여 네트워크 정보 관리

예를 들어 사용자가 Network를 생성하는 경우 neutron-server가 해당 요청을 처리하고 선택된 Plugin과 연동한다.

Neutron API의 접근 주소는 관리 목적에 맞게 지정할 수 있으며, 내부 관리 네트워크 또는 API 네트워크를 통해 제공할 수 있다.

10.2 Neutron DHCP Agent

`neutron-dhcp-agent`는 Neutron Network에서 DHCP 서비스를 제공하도록 관리하는 구성 요소이다.

DHCP는 VM이 네트워크 설정을 자동으로 획득할 수 있도록 하는 프로토콜이다.

대표적인 설정 정보는 다음과 같다.

- IP Address
- Subnet Mask
- Default Gateway
- DNS Server

Neutron DHCP Agent는 **dnsmasq** 프로세스를 생성하고 관리하여 DHCP 기능을 제공한다.

dnsmasq

dnsmasq는 가벼운 DNS Forwarder 및 DHCP Server이다.

Neutron에서는 DHCP를 사용하는 Network에 대해 dnsmasq 프로세스를 실행하여 DHCP 요청을 처리할 수 있다.

일반적인 DHCP 동작 흐름은 다음과 같다.

```
VM
 |
 | DHCP Request
 v
Network
 |
 v
DHCP Namespace
 |
dnsmasq
 |
 | DHCP Response
 v
VM
```

VM에서 DHCP 요청이 발생하면 해당 네트워크의 DHCP 서비스가 요청을 처리하고 필요한 네트워크 설정을 제공한다.

DHCP Agent와 Namespace

DHCP Agent는 DHCP 서비스의 격리를 위해 Network Namespace를 활용할 수 있다.

예를 들어 다음과 같은 Namespace가 존재할 수 있다.

```
qdhcp-<network UUID>
```

해당 Namespace에는 DHCP 서비스가 사용하는 네트워크 인터페이스와 dnsmasq 프로세스가 구성된다.

이를 통해 서로 다른 Network의 DHCP 서비스를 독립적으로 실행할 수 있다.

DHCP Lease와 Neutron Port의 IP 할당

DHCP Lease는 DHCP Client가 할당받은 IP 주소를 사용할 수 있는 임대 시간을 의미한다.

그러나 DHCP Lease가 만료되는 시점과 Neutron Port에 할당된 IP 주소가 해제되는 시점은 동일하지 않다.

Neutron이 Port에 IP 주소를 할당한 경우, DHCP Lease가 만료되거나 VM이 종료되었다는 이유만으로 해당 Port의 IP 할당이 즉시 사라지는 것은 아니다.

Port의 IP 할당은 Port 또는 관련 리소스의 삭제 등의 동작에 따라 관리된다.

따라서 **DHCP가 VM에 설정을 전달하는 기능과 Neutron이 Port의 IP 주소 할당 정보를 관리하는 기능은 구분해야 한다.**

10.3 Neutron Metadata Agent

Metadata는 VM이 자신의 인스턴스 정보를 조회할 수 있도록 제공되는 데이터이다.

대표적인 정보는 다음과 같다.

- Hostname
- SSH Public Key
- 인스턴스 관련 설정 정보

Nova는 인스턴스에 이러한 정보를 제공하기 위한 Metadata Service를 지원한다.

VM은 일반적으로 다음 주소를 통해 Metadata를 요청한다.

```
http://169.254.169.254
```

Metadata Agent의 역할

`neutron-metadata-agent`는 VM의 Metadata 요청을 Nova Metadata API로 전달하는 역할을 수행한다.

VM의 Metadata 요청은 Network Namespace에 존재하는 Metadata Proxy를 통해 전달될 수 있다.

대표적인 동작 흐름은 다음과 같다.

```
VM
 |
 | HTTP Request
 | 169.254.169.254
 v
Router / DHCP Namespace
 |
 v
Metadata Proxy
 |
 v
Neutron Metadata Agent
 |
 v
Nova Metadata API
```

응답은 반대 방향으로 전달된다.

Metadata 요청 과정

1. VM이 부팅 중 Metadata 주소로 HTTP 요청을 전송한다.
2. 해당 요청이 Router 또는 DHCP Namespace의 Metadata Proxy에 도착한다.
3. Metadata Proxy가 Neutron Metadata Agent로 요청을 전달한다.
4. Neutron Metadata Agent가 Nova Metadata API로 요청을 전달한다.
5. Nova Metadata API가 인스턴스 정보를 반환한다.
6. Metadata Agent가 응답을 Metadata Proxy로 전달한다.
7. Metadata Proxy가 VM에 응답을 전달한다.
8. VM은 반환된 Metadata를 이용하여 부팅 과정의 설정을 진행할 수 있다.

Metadata Proxy와 Metadata Agent 사이의 통신에는 Unix Socket을 사용할 수 있다.

Router를 사용하는 경우

Neutron Router가 VM의 기본 게이트웨이 역할을 하는 경우에는 Router Namespace를 통한 Metadata 접근 경로를 사용할 수 있다.

Router를 사용하지 않는 경우

별도의 물리 라우터가 VM의 기본 게이트웨이 역할을 하는 경우에도 Neutron DHCP 서비스를 통해 Metadata 접근 경로를 제공할 수 있다.

이러한 경우에는 DHCP를 이용하여 VM에 Metadata 서비스로 향하는 경로 정보를 전달할 수 있다.

따라서 Metadata 요청이 반드시 Neutron Router를 거쳐야 하는 것은 아니다.

Metadata Agent는 VM에 IP 주소를 할당하는 서비스가 아니라, VM이 인스턴스 정보를 조회할 수 있도록 요청을 전달하는 서비스이다.

10.4 Neutron L3 Agent

`neutron-l3-agent`는 Neutron Router를 구성하고 관리하는 서비스이다.

Neutron Router는 서로 다른 L2 네트워크를 연결하고, 필요에 따라 외부 네트워크와의 연결을 제공한다.

주요 기능은 다음과 같다.

- 가상 라우터 구성
- 네트워크 간 Routing
- IP Forwarding
- NAT
- Floating IP 지원

L3 Agent는 Linux IP Stack과 iptables 등을 활용하여 이러한 기능을 구현한다.

Router Namespace

서로 다른 Tenant가 독립적인 Router를 사용할 수 있도록 L3 Agent는 Network Namespace를 활용한다.

대표적인 형태는 다음과 같다.

```
qrouter-<router UUID>
```

각 Router Namespace는 독립적인 Routing Table과 네트워크 인터페이스 등을 가질 수 있다.

이를 통해 하나의 Linux 호스트에서 여러 가상 라우터를 실행할 수 있다.

내부 네트워크 간 Routing

예를 들어 Router가 두 개의 내부 네트워크에 연결되어 있다면 다음과 같이 패킷을 전달할 수 있다.

```
Network A
192.168.10.0/24
      |
      v
Neutron Router
      |
      v
Network B
192.168.20.0/24
```

Router는 연결된 네트워크 사이에서 패킷을 전달한다.

외부 네트워크 연결

Neutron Router가 External Network에 연결되면 내부 VM의 외부 네트워크 접근을 지원할 수 있다.

필요한 경우 NAT를 사용하여 내부 사설 IP와 외부 주소 사이의 변환을 수행한다.

Floating IP

Floating IP는 외부 네트워크의 IP 주소를 내부 VM의 Port에 연결하기 위한 기능이다.

이를 통해 외부 네트워크에서 내부 VM에 접근하는 경로를 구성할 수 있다.

L3 Agent는 이러한 Router 및 NAT 구성을 관리하는 역할을 담당한다.

10.5 Neutron LBaaS Agent

`neutron-lbaas-agent`는 초기 Neutron LBaaS의 가상 로드밸런서를 구성하고 관리하는 서비스이다.

Neutron API를 통해 Load Balancer를 생성하면 LBaaS Agent가 해당 로드밸런서 기능을 구현한다.

Havana와 Icehouse의 참조 구현에서는 HAProxy가 사용된다.

HAProxy

HAProxy는 클라이언트의 요청을 여러 백엔드 서버로 분산하는 소프트웨어 로드밸런서이다.

예를 들어 다음과 같은 구조를 만들 수 있다.

```
Client
   |
   v
Virtual IP
   |
 HAProxy
   |
   +---- VM1
   |
   +---- VM2
   |
   +---- VM3
```

LBaaS Agent는 Network Namespace를 이용하여 로드밸런서의 네트워크 환경을 격리할 수 있다.

이러한 방식으로 여러 가상 로드밸런서를 서로 독립적으로 실행할 수 있다.

---

11. Neutron 서비스 간 관계

Neutron의 네트워크 관리 기능은 여러 구성요소가 협력하여 제공한다.

각 서비스의 역할을 정리하면 다음과 같다.

구성요소	주요 역할	관련 네트워크 기능
neutron-server	API 요청 및 네트워크 구성 관리	Network, Subnet, Port
Network Plugin / Agent	가상 네트워크 연결 구성	L2 Switching
DHCP Agent	DHCP 서비스 관리	IP 및 네트워크 설정 제공
Metadata Agent	VM Metadata 요청 전달	인스턴스 정보 조회
L3 Agent	Router 구성 및 관리	Routing, NAT, Floating IP
LBaaS Agent	가상 로드밸런서 관리	Load Balancing

11.1 VM 생성과 네트워크 연결

VM이 생성되어 네트워크를 사용하는 과정을 연결하면 다음과 같다.

```
User
 |
 | VM 생성 요청
 v
Nova
 |
 | Neutron Port / Network 사용
 v
Neutron
 |
 +--- Network Plugin / Agent
 |         |
 |         v
 |     가상 네트워크 연결
 |
 +--- DHCP Agent
 |         |
 |         v
 |     네트워크 설정 제공
 |
 +--- Metadata Agent
 |         |
 |         v
 |     Nova Metadata API
 |
 +--- L3 Agent
           |
           v
       Router / NAT
```

각 서비스는 독립적인 역할을 수행하며, VM이 실제로 네트워크를 사용하기 위해서는 필요한 구성요소들이 함께 동작해야 한다.

11.2 서비스별 장애 영향

네트워크 서비스를 기능별로 구분하면 장애 발생 시 영향을 받는 범위를 추정할 수 있다.

구성요소	문제가 발생했을 때 예상할 수 있는 영향
neutron-server	새로운 네트워크 구성 요청 실패
Network Plugin / Agent	네트워크 연결 및 구성 변경 문제
DHCP Agent	VM의 DHCP 설정 획득 실패
Metadata Agent	VM의 인스턴스 정보 조회 실패
L3 Agent	Router 및 NAT 구성 관리 문제

구성요소	문제가 발생했을 때 예상할 수 있는 영향
LBaaS Agent	로드밸런서 구성 관리 문제

이 표는 각 구성요소의 기능에 따른 예상 영향이다.

실제 장애 범위는 기존 네트워크 설정 상태, 이미 실행 중인 프로세스, 네트워크 연결 방식 등에 따라 달라질 수 있다.

예를 들어 DHCP Agent가 정상적으로 실행되지 않는다고 해서 기존에 IP 주소를 가지고 통신 중인 모든 VM의 네트워크가 즉시 끊어지는 것은 아니다.

반대로 신규 VM이 DHCP 설정을 획득해야 하는 경우에는 영향을 받을 수 있다.

따라서 특정 구성요소의 상태뿐 아니라 해당 구성요소가 제공하는 실제 기능을 구분하여 이해해야 한다.

---

12. Chapter 1 & 3 종합 정리

12.1 네트워크 구성의 기본 요소

Neutron은 Network, Subnet, Port 등의 리소스를 이용하여 가상 네트워크를 정의한다.

- Network: 논리적인 L2 네트워크
- Subnet: IP 주소 범위
- Port: 네트워크 인터페이스의 논리적 연결 지점

이러한 리소스는 Neutron API를 통해 관리된다.

12.2 네트워크 기능

OpenStack Networking은 가상화된 환경에서 다양한 네트워크 기능을 제공한다.

- Switching: 동일한 L2 네트워크 내부의 프레임 전달
- Routing: 서로 다른 IP 네트워크 사이의 패킷 전달
- Load Balancing: 여러 서버로 트래픽 분산
- Firewalling: 보안 규칙 적용
- VPN: 원격 네트워크 사이의 논리적 연결

12.3 네트워크 격리

Linux Network Namespace는 독립적인 네트워크 환경을 제공한다.

이를 이용하여 하나의 Linux 호스트에서 여러 Tenant의 DHCP, Router, Load Balancer 등의 네트워크 서비스를 격리하여 실행할 수 있다.

12.4 물리 네트워크

OpenStack의 가상 네트워크는 물리 네트워크 인프라 위에서 동작한다.

Management, API, External, Guest Traffic은 목적에 따라 구분되며, 하나 또는 여러 물리 NIC를 사용하도록 구성할 수 있다.

Single Interface, Multiple Interfaces, Bonding은 서로 다른 물리 연결 설계 방법이다.

12.5 노드 역할

OpenStack의 서비스는 Controller, Network, Compute Node에 나누어 배치할 수 있다.

- Controller: OpenStack 관리 및 API 서비스
- Network: DHCP, Metadata, Router 등 네트워크 서비스
- Compute: VM 실행 및 가상 네트워크 연결

서비스의 배치 위치는 네트워크 성능과 장애 영향 범위에 영향을 준다.

12.6 Neutron 서비스

Neutron은 여러 구성요소가 협력하여 동작한다.

`neutron-server`는 API 요청을 처리하며, Network Plugin과 Agent는 선택한 네트워크 기술 및 네트워크 서비스의 구현을 담당한다.

DHCP, Metadata, L3 Agent는 서로 다른 기능을 담당하기 때문에 각각의 역할과 통신 흐름을 구분하여 이해해야 한다.

---

13. 학습 확인 질문

Chapter 1

1. Neutron은 OpenStack 환경에서 어떤 역할을 수행하는가?
2. Software-Defined Networking을 이용하면 네트워크 관리 방식이 어떻게 달라지는가?
3. Switching과 Routing의 차이는 무엇인가?

4. Linux Bridge와 Open vSwitch는 어떤 역할을 수행하는가?
5. VLAN과 VXLAN은 각각 어떤 방식으로 네트워크를 구성하는가?
6. Management, API, External, Guest Network는 어떤 목적을 가지는가?
7. Single Interface 구성에서 Guest Traffic이 관리 통신에 영향을 줄 수 있는 이유는 무엇인가?
8. Multiple Interfaces와 Bonding의 목적은 어떻게 다른가?
9. Controller에 L3 Agent를 배치하는 방식과 전용 Network Node에 배치하는 방식은 어떤 차이가 있는가?

Chapter 3

1. Network, Subnet, Port는 각각 무엇을 의미하며 어떤 관계를 가지는가?
2. 서로 다른 Tenant가 동일한 사설 IP 대역을 사용하려면 어떤 네트워크 격리 기능이 필요한가?
3. `qdhcp`, `qrouter`, `qlbaas` Namespace는 각각 어떤 역할을 수행하는가?
4. Network Plugin은 Neutron API와 실제 네트워크 구현 기술 사이에서 어떤 역할을 수행하는가?
5. neutron-server와 Network Agent의 역할은 어떻게 다른가?
6. DHCP Agent와 Metadata Agent는 어떤 차이가 있는가?
7. VM에서 `169.254.169.254`로 요청을 전송하면 어떤 구성요소를 거쳐 Nova Metadata API에 도달하는가?
8. L3 Agent는 어떤 Linux 기능을 이용하여 Router를 구현하는가?
9. DHCP Lease가 만료되는 것과 Neutron Port의 IP 주소 할당이 해제되는 것은 같은 의미인가?

최종 학습 목표: Neutron의 기본 리소스와 네트워크 서비스의 역할을 이해하고, VM의 네트워크 연결·IP 설정·Metadata 요청·라우팅이 어떤 구성요소와 관계되는지 설명할 수 있어야 한다.

Chapter 6. Monitoring Distributed Systems

Site Reliability Engineering: How Google Runs Production Systems

- 저자: Rob Ewaschuk
- 편집: Betsy Beyer
- 학습 범위: PDF p.81–92 (인쇄 p.55–66)

1. Definitions

Monitoring은 시스템에서 발생하는 데이터를 수집하고 분석하여 시스템의 상태를 파악하는 활동이다.

운영 환경에서는 다양한 구성요소가 함께 동작하기 때문에 단순히 프로세스의 실행 여부만으로 서비스가 정상적으로 동작하는지 판단하기 어렵다.

따라서 서비스의 요청량, 오류, 처리 시간, 시스템 자원 사용량 등을 관찰해야 한다.

1.1 Monitoring

Monitoring은 시스템에서 발생하는 실시간 정량 데이터를 수집, 처리, 집계, 표시하는 과정이다.

대표적인 모니터링 데이터는 다음과 같다.

- 요청 수와 요청 유형
- 오류 발생 수와 오류 유형
- 요청 처리 시간
- 서버 및 프로세스 실행 시간
- 시스템 자원 사용량

이러한 데이터를 통해 현재 시스템의 상태를 파악하고 시간에 따른 변화를 분석할 수 있다.

1.2 White-box Monitoring

White-box Monitoring은 시스템 내부에서 제공되는 정보를 기반으로 상태를 관찰하는 방식이다.

시스템 내부에 접근할 수 있다는 전제하에 로그, 내부 지표, 상태 조회 인터페이스 등을 사용한다.

대표적인 관찰 대상은 다음과 같다.

- 애플리케이션 로그
- 내부 오류 카운터

- 프로세스 상태
- 서비스가 노출하는 성능 지표
- 내부 API 및 모니터링 엔드포인트

White-box Monitoring은 단순히 서비스가 정상인지 확인하는 것뿐 아니라 시스템 내부에서 어떤 일이 발생하는지 이해하는 데 활용된다.

1.3 Black-box Monitoring

Black-box Monitoring은 시스템 외부에서 사용자가 경험하는 동작을 기준으로 서비스를 확인하는 방식이다.

서비스 내부의 구현 방식이나 구성요소를 직접 확인하지 않고 외부에서 요청을 보내 결과를 관찰한다.

예를 들어 다음과 같은 확인이 가능하다.

- 웹 서비스에 HTTP 요청을 전송하여 응답 확인
- 서비스 응답 시간 측정
- 외부에서 서비스에 접근할 수 있는지 확인
- 요청한 결과가 올바르게 반환되는지 검증

Black-box Monitoring은 **사용자가 실제로 서비스를 정상적으로 사용할 수 있는지** 확인하는 데 적합하다.

1.4 Dashboard

Dashboard는 서비스의 핵심 지표를 한눈에 확인할 수 있도록 구성한 화면이다.

대표적인 표시 항목은 다음과 같다.

- 서비스 요청량
- 오류 발생률
- 응답 시간
- 자원 사용량
- 현재 발생한 주요 문제
- 운영 관련 상태 정보

Dashboard는 서비스의 현재 상태를 파악하거나 장애를 조사할 때 활용한다.

단순히 모든 지표를 나열하는 것보다 운영자가 알아야 할 핵심 정보를 제공하는 것이 중요하다.

1.5 Alert

Alert는 시스템 상태에 대한 문제를 사람에게 알리는 통지이다.

대표적인 알림 방식은 다음과 같다.

종류	설명
Ticket	작업이나 문제를 추적하기 위해 티켓 또는 이슈로 등록
Email Alert	이메일을 통해 상태나 문제를 알림
Page	운영자의 즉각적인 주의와 대응을 요구하는 알림

세 가지 방식은 모두 Alert에 해당하지만 운영자에게 요구하는 대응 수준이 다르다.

특히 Page는 운영자의 업무와 휴식 시간을 중단시킬 수 있기 때문에 신중하게 사용해야 한다.

1.6 Root Cause

Root Cause는 장애를 발생시킨 근본적인 결함이나 원인이다.

단순히 서비스를 재시작하여 장애가 일시적으로 해결되었다고 해서 Root Cause가 제거된 것은 아니다.

예를 들어 어떤 애플리케이션이 잘못된 설정으로 인해 반복적으로 종료되는 상황을 생각할 수 있다.

프로세스를 재시작하면 서비스는 일시적으로 정상 상태로 돌아올 수 있지만, 잘못된 설정이 유지되면 동일한 문제가 다시 발생할 수 있다.

따라서 장애의 원인을 찾아 재발 가능성을 줄이는 것이 중요하다.

또한 하나의 장애에는 여러 Root Cause가 존재할 수 있다.

예를 들어 잘못된 설정 파일, 부족한 자동화, 검증되지 않은 변경 과정이 함께 장애를 발생시킬 수 있다.

1.7 Node, Machine, Push

Node / Machine

하나의 실행 중인 운영체제 커널 인스턴스를 의미한다. 물리 서버뿐 아니라 VM이나 컨테이너 환경도 포함될 수 있다.

하나의 Machine에서 여러 서비스가 실행될 수 있으며, 각 서비스를 독립적으로 모니터링해야 할 수 있다.

Push

실행 중인 서비스의 소프트웨어 또는 설정을 변경하는 행위를 의미한다.

새로운 버전 배포뿐 아니라 설정 변경도 Push에 포함된다.

서비스에 문제가 발생했을 때 최근 Push가 있었는지 확인하는 것은 장애 원인을 분석하는 데 중요한 단서가 될 수 있다.

---

2. Why Monitor?

Monitoring의 목적은 단순히 시스템이 정상적으로 실행되고 있는지 확인하는 것만이 아니다.

수집된 데이터를 활용하여 장기적인 변화를 파악하고, 성능을 비교하고, 장애를 탐지하며, 문제 발생 시 원인을 분석할 수 있다.

2.1 Analyzing Long-Term Trends

장기간 수집한 데이터를 이용하여 시스템의 변화 추세를 분석한다.

예를 들어 다음과 같은 질문에 답할 수 있다.

- 데이터베이스 크기가 얼마나 증가하고 있는가?
- 사용자 수가 얼마나 빠르게 증가하는가?
- 시스템의 자원 사용량이 지속적으로 증가하고 있는가?

장기적인 추세를 파악하면 향후 필요한 자원을 예측하고 시스템 용량을 계획할 수 있다.

2.2 Comparing Over Time or Experiment Groups

서로 다른 시점이나 설정에서 시스템의 성능을 비교할 수 있다.

예를 들어 다음과 같은 상황이다.

- 새로운 버전 적용 전후의 응답 시간 비교
- 캐시 서버 추가 전후의 성능 비교
- 이전 주와 현재 주의 요청 처리 속도 비교

이러한 비교를 통해 설정 변경이나 시스템 개선이 실제로 효과가 있었는지 판단할 수 있다.

2.3 Alerting

Monitoring은 시스템에 문제가 발생했거나 곧 문제가 발생할 가능성이 있는 경우 운영자에게 알림을 제공할 수 있다.

예를 들어 서비스의 오류율이 급격하게 증가하는 경우 운영자가 문제를 인지하고 대응할 수 있도록 한다.

그러나 모든 비정상적인 변화가 즉각적인 알림을 필요로 하는 것은 아니다.

Alert는 사람이 확인하고 대응해야 할 의미 있는 문제를 전달하기 위한 수단이다.

2.4 Building Dashboards

Dashboard는 운영자가 서비스의 주요 지표를 빠르게 확인할 수 있도록 한다.

이를 통해 시스템의 전체 상태와 주요 변화를 파악할 수 있다.

특히 서비스 요청량, 오류, 지연 시간, 자원 포화도 등의 핵심 지표가 중요하다.

2.5 Conducting Ad Hoc Retrospective Analysis

문제가 발생했을 때 과거 데이터를 조회하여 원인을 분석할 수 있다.

예를 들어 서비스 응답 시간이 갑자기 증가했다면 같은 시점에 CPU 사용량, 오류율, 요청량 등이 어떻게 변화했는지 비교할 수 있다.

이를 통해 여러 지표 사이의 연관성을 분석할 수 있다.

2.6 Monitoring과 Alerting의 관계

Monitoring은 시스템의 상태를 파악하기 위해 데이터를 지속적으로 수집하고 분석하는 과정이다.

Alerting은 수집된 데이터를 기반으로 사람이 대응해야 할 문제를 알려주는 기능이다.

즉, Monitoring을 수행한다고 해서 모든 문제에 대해 즉시 Alert를 발생시킬 필요는 없다.

알림을 받은 사람은 문제를 확인하고, 필요한 경우 영향을 완화한 뒤 근본 원인을 조사해야 한다.

운영자를 호출하는 것은 비용이 큰 행위이다.

잦은 알림은 업무를 중단시키고 피로도를 증가시키며, 실제로 중요한 알림까지 무시하게 만들 수 있다.

따라서 효과적인 Alerting 시스템은 **유용한 신호는 많고 불필요한 알림은 적어야 한다.**

---

3. Setting Reasonable Expectations for Monitoring

복잡한 분산 시스템을 모니터링하는 것은 그 자체로 상당한 엔지니어링 작업이다.

시스템에서 필요한 데이터를 수집하고 저장하며, 이를 분석하여 적절한 알림을 발생시키기 위한 기능을 설계해야 한다.

그러나 모니터링 시스템이 지나치게 복잡해지면 오히려 운영하기 어려워질 수 있다.

3.1 Monitoring 시스템의 복잡성

모니터링 시스템에는 다음과 같은 기능이 포함될 수 있다.

- 데이터 수집
- 데이터 집계
- 시계열 데이터 저장
- Dashboard 제공
- Alert 규칙 평가
- 운영자 알림 전달

이러한 기능이 많아질수록 모니터링 시스템 자체의 유지보수 부담도 증가할 수 있다.

따라서 모든 상황을 자동으로 분석하고 원인을 추론하는 복잡한 시스템을 만드는 것이 항상 좋은 선택은 아니다.

3.2 단순하고 이해하기 쉬운 Monitoring

장애 탐지부터 운영자의 대응까지 이어지는 과정은 단순하고 이해하기 쉬워야 한다.

일반적인 흐름은 다음과 같다.

```
서비스 문제 발생
       |
       v
Monitoring
       |
       v
Alert 발생
       |
       v
운영자 문제 확인
       |
       v
초기 대응 및 조사
       |
       v
원인 분석
```

이 과정에서 복잡한 규칙이 지나치게 많다면 Alert가 발생한 이유를 이해하기 어려워질 수 있다.

특히 즉각적인 운영자 대응을 요구하는 Page는 단순하고 신뢰할 수 있는 규칙을 기반으로 발생해야 한다.

3.3 자동 이상 탐지의 한계

복잡한 시스템에서는 정상 상태와 비정상 상태를 자동으로 구분하는 것이 항상 쉽지 않다.

예를 들어 요청량이 급격하게 증가한 상황은 다음과 같은 이유로 발생할 수 있다.

- 실제 사용자 증가
- 정상적인 이벤트나 캠페인
- 비정상적인 트래픽
- 시스템의 오류

따라서 단순히 평소와 다른 지표가 발생했다는 이유만으로 문제가 있다고 확정하기 어렵다.

자동 이상 탐지 기능을 사용할 수는 있지만, 운영자가 이해할 수 있는 명확한 조건과 목적을 갖는 것이 중요하다.

3.4 복잡한 의존성 규칙의 문제

서비스는 여러 구성요소에 의존하여 동작한다.

예를 들어 웹 서비스가 데이터베이스에 의존하는 경우 데이터베이스 장애가 웹 서비스 장애로 이어질 수 있다.

그러나 모든 구성요소의 의존성 관계를 Alert 규칙에 복잡하게 반영하면 설정과 유지보수가 어려워질 수 있다.

특히 분산 시스템의 구조는 지속적으로 변경되기 때문에 의존성 규칙도 함께 수정해야 한다.

따라서 모니터링 시스템은 변경되는 구조를 고려하여 가능한 한 단순한 방식으로 설계하는 것이 바람직하다.

핵심은 사람이 즉시 대응해야 하는 문제를 명확하고 신뢰할 수 있는 방식으로 탐지하는 것이다.

---

4. Symptoms Versus Causes

모니터링 시스템은 크게 두 가지 질문에 답할 수 있어야 한다.

1. **What is broken? — 무엇이 잘못되었는가?**
2. **Why is it broken? — 왜 문제가 발생했는가?**

첫 번째 질문은 Symptom(증상)에 해당한다.

두 번째 질문은 Cause(원인)에 해당한다.

4.1 Symptom

Symptom은 서비스가 정상적으로 동작하지 않는 상태를 나타내는 현상이다.

대표적인 예시는 다음과 같다.

- 웹 서비스의 HTTP 오류 증가
- 서비스 응답 시간 증가
- 사용자가 서비스에 접근하지 못함
- 요청한 데이터가 정상적으로 반환되지 않음

Symptom은 주로 사용자 관점에서 관찰할 수 있는 문제와 관련된다.

4.2 Cause

Cause는 Symptom이 발생하게 된 원인이다.

대표적인 예시는 다음과 같다.

- 데이터베이스 연결 실패
- CPU 과부하
- 네트워크 패킷 손실
- 잘못된 설정 변경
- 접근 제어 정책 오류

예를 들어 웹 서비스의 응답이 느려졌다는 것은 Symptom이다.

그 원인은 CPU 사용량 증가, 데이터베이스 처리 지연 또는 네트워크 패킷 손실 등 여러 가지일 수 있다.

4.3 Symptom과 Cause의 관계

Symptom	가능한 Cause
HTTP 500 오류 발생	데이터베이스 연결 실패
서비스 응답 시간 증가	CPU 과부하 또는 네트워크 패킷 손실
특정 사용자에게 콘텐츠 제공 실패	접근 제한 또는 네트워크 정책 문제
비공개 데이터에 외부 접근 가능	잘못된 접근 제어 설정

하나의 증상에는 여러 원인이 존재할 수 있으므로, 증상만으로 원인을 확정할 수 없다.

반대로 하나의 원인이 여러 증상을 발생시킬 수도 있다.

예를 들어 데이터베이스 장애는 응답 시간 증가, HTTP 오류 및 일부 기능 실패를 동시에 발생시킬 수 있다.

4.4 Monitoring에서 두 개념을 구분해야 하는 이유

효과적인 모니터링 시스템을 만들기 위해서는 사용자에게 발생한 문제와 내부 구성요소에서 발생한 문제를 구분해야 한다.

사용자에게 이미 장애가 발생했다면 실제 증상을 빠르게 발견하는 것이 중요하다.

반면 장애의 원인을 조사하거나 앞으로 발생할 문제를 예상하려면 내부 지표가 필요하다.

따라서 Symptom과 Cause를 구분하는 것은 Monitoring 및 Alerting 설계의 중요한 기준이다.

---

5. Black-Box Versus White-Box

Black-box Monitoring과 White-box Monitoring은 시스템을 관찰하는 서로 다른 방식이다.

두 방식은 서로 대체하는 관계가 아니라 상호 보완적인 관계이다.

5.1 Black-box Monitoring

Black-box Monitoring은 서비스 외부에서 실제 사용자처럼 요청을 수행하여 결과를 확인한다.

예를 들어 웹 서비스에 HTTP 요청을 보내고 정상적인 응답이 반환되는지 확인할 수 있다.

```
Monitoring Client
       |
       | HTTP Request
       v
Web Service
       |
       | HTTP Response
       v
Monitoring Client
```

이 방식은 서비스 내부의 구현과 관계없이 사용자에게 보이는 동작을 확인한다.

주요 특징

- 외부에서 서비스의 동작을 확인
- 사용자 관점의 문제를 탐지
- 실제 서비스 접근 가능 여부 확인
- 이미 발생한 장애를 발견하는 데 유용

한계

Black-box Monitoring은 서비스가 현재 정상적으로 응답하더라도 내부에서 발생 중인 잠재적인 문제를 발견하지 못할 수 있다.

예를 들어 시스템의 디스크 사용량이 95%에 도달했더라도 서비스 요청이 정상적으로 처리되고 있다면 Black-box Monitoring만으로는 문제를 알기 어려울 수 있다.

5.2 White-box Monitoring

White-box Monitoring은 시스템 내부의 상태와 지표를 수집하여 동작을 확인한다.

대표적인 확인 항목은 다음과 같다.

- CPU 및 Memory 사용량
- 내부 오류 카운터
- 애플리케이션 처리 시간
- 로그
- 내부 요청 실패
- 재시도 횟수

내부 상태를 관찰할 수 있기 때문에 실제 사용자 장애가 발생하기 전의 이상 징후를 발견할 수 있다.

주요 특징

- 내부 구성요소의 상태 확인
- 문제 원인 분석에 활용
- 잠재적인 장애 탐지
- 재시도 등에 의해 외부에서 보이지 않는 문제 확인

한계

내부 구성요소가 정상 상태라고 해서 실제 사용자 요청이 정상적으로 처리된다는 보장은 없다.

예를 들어 웹 서버 프로세스가 실행 중이고 CPU 사용량이 정상이라도, 네트워크 문제로 인해 사용자가 웹 서버에 접근하지 못할 수 있다.

5.3 다계층 시스템에서의 Symptom과 Cause

복잡한 시스템에서는 한 구성요소의 Symptom이 다른 구성요소의 Cause가 될 수 있다.

예를 들어 다음과 같은 서비스 구조를 생각할 수 있다.

```
User
 |
 v
Web Server
 |
 v
Database
```

Database의 응답 시간이 증가하는 문제가 발생했다고 가정한다.

Database를 관리하는 운영자에게는 Database의 처리 지연이 Symptom이다.

반면 Web Server를 관리하는 운영자에게는 Database의 처리 지연이 웹 서비스 응답 시간을 증가시키는 Cause가 될 수 있다.

따라서 Symptom과 Cause는 어떤 서비스와 관점에서 문제를 바라보는지에 따라 달라질 수 있다.

5.4 White-box Monitoring을 이용한 원인 분석

Web Server의 응답 시간이 증가했을 때 다음 두 지표를 비교할 수 있다.

- Web Server가 관찰한 Database 요청 처리 시간
- Database가 내부적으로 기록한 쿼리 처리 시간

Web Server에서는 Database 요청이 느리다고 판단하지만 Database 내부 처리 시간이 정상이라면, 두 서비스 사이의 네트워크 구간 등에 문제가 있을 가능성을 조사할 수 있다.

이처럼 여러 구성요소에서 관찰한 지표를 비교하면 장애 범위를 줄일 수 있다.

5.5 두 Monitoring 방식의 비교

항목	Black-box	White-box
관찰 위치	시스템 외부	시스템 내부
주요 목적	실제 서비스 동작 확인	내부 상태 확인 및 원인 분석
사용자 장애 탐지	적합	내부 지표에 따라 다름
잠재적인 문제 탐지	제한적	적합
대표적인 예시	HTTP 요청 테스트	CPU, 로그, 오류 카운터

실제 사용자에게 영향을 주는 장애를 탐지하는 데는 Black-box Monitoring이 중요하다.

반면 장애의 원인을 분석하고 장애가 발생하기 전에 문제를 발견하려면 White-box Monitoring이 필요하다.

6. The Four Golden Signals

Google SRE에서는 사용자에게 서비스를 제공하는 시스템을 모니터링할 때 네 가지 핵심 지표를 강조한다.

이를 **Four Golden Signals**라고 한다.

1. Latency
2. Traffic
3. Errors
4. Saturation

이 네 가지는 사용자 관점의 서비스 상태를 파악하기 위한 기본적인 출발점이다.

6.1 Latency

Latency는 요청을 처리하는 데 걸리는 시간이다.

예를 들어 사용자가 웹 서비스에 요청을 전송한 시점부터 응답을 받는 시점까지 걸리는 시간을 측정할 수 있다.

일반적으로 Latency가 증가하면 사용자 경험이 나빠질 수 있다.

그러나 모든 요청의 처리 시간을 동일하게 취급하면 실제 시스템 상태를 잘못 판단할 수 있다.

성공한 요청과 실패한 요청

성공한 요청과 실패한 요청의 Latency를 구분하는 것이 중요하다.

예를 들어 다음과 같은 상황을 가정한다.

요청	결과	응답 시간
요청 A	HTTP 200	300ms
요청 B	HTTP 200	350ms
요청 C	HTTP 500	5ms

요청 C는 서버 내부 오류가 발생했지만 응답 시간이 매우 짧다.

전체 요청의 평균 응답 시간만 측정하면 빠르게 실패한 요청으로 인해 평균 Latency가 낮아질 수 있다.

이 경우 실제 서비스는 장애 상태이지만 평균 응답 시간은 개선된 것처럼 보일 수 있다.

따라서 성공한 요청과 실패한 요청의 Latency를 구분하여 측정해야 한다.

또한 실패한 요청이라고 해서 Latency 측정에서 완전히 제외하는 것도 적절하지 않다.

오랜 시간이 지난 후 실패하는 요청 역시 사용자 경험에 부정적인 영향을 주기 때문이다.

6.2 Traffic

Traffic은 시스템에 전달되는 요청이나 작업의 양을 의미한다.

단순히 네트워크 인터페이스의 전송량만을 의미하는 것은 아니다.

서비스의 유형에 따라 적절한 지표를 선택해야 한다.

서비스	Traffic 지표 예시
Web Service	초당 HTTP 요청 수
Streaming Service	네트워크 전송량 또는 동시 세션 수
Key-Value Storage	초당 트랜잭션 및 조회 수

예를 들어 웹 서비스에서는 초당 요청 수(Requests per Second)가 중요한 지표가 될 수 있다.

Traffic을 관찰하면 서비스가 처리해야 하는 부하가 얼마나 증가하고 있는지 파악할 수 있다.

또한 요청량이 급격하게 증가하거나 감소하는 상황을 발견하는 데 활용할 수 있다.

6.3 Errors

Errors는 실패한 요청의 비율 또는 발생량을 의미한다.

오류는 단순히 서버에서 반환한 명시적인 에러 코드만을 의미하지 않는다.

세 가지 형태로 구분할 수 있다.

Explicit Errors — 명시적 오류

서버가 명확한 오류 코드를 반환하는 경우이다.

예를 들어 HTTP 500 응답은 서버에서 요청을 정상적으로 처리하지 못했음을 나타낸다.

Implicit Errors — 암묵적 오류

서버는 성공 응답을 반환했지만 실제 결과가 잘못된 경우이다.

예를 들어 서버가 HTTP 200을 반환했지만 사용자에게 잘못된 데이터를 제공했다면, 상태 코드만으로는 오류를 발견할 수 없다.

Policy Errors — 정책상 오류

요청 자체는 성공했지만 정해진 서비스 품질 기준을 충족하지 못한 경우이다.

예를 들어 서비스가 1초 이내 응답을 목표로 하는데 요청 처리 시간이 3초라면, 요청은 성공했어도 서비스 운영 기준에 따라 오류로 판단할 수 있다.

따라서 오류를 측정할 때는 단순히 HTTP 상태 코드뿐 아니라 서비스의 정상 동작 기준을 함께 고려해야 한다.

Error Rate

전체 요청 중 실패한 요청이 차지하는 비율을 Error Rate라고 한다.

예를 들어 총 1,000개의 요청 중 20개가 실패했다면 다음과 같이 계산한다.

```
Error Rate = 실패 요청 수 / 전체 요청 수 × 100

           = 20 / 1000 × 100
           = 2%
```

Error Rate는 사용자 요청이 얼마나 안정적으로 처리되고 있는지 파악하는 데 사용된다.

6.4 Saturation

Saturation은 시스템의 제한적인 자원이 얼마나 사용되고 있는지를 나타내는 지표이다.

특히 시스템에서 가장 제약이 큰 자원을 중심으로 관찰한다.

대표적인 자원은 다음과 같다.

- CPU
- Memory
- Disk
- Network Bandwidth
- I/O 처리 능력

예를 들어 Memory가 제한적인 서비스라면 Memory 사용량이 중요한 Saturation 지표가 될 수 있다.

네트워크 대역폭이 병목인 시스템에서는 Network Bandwidth가 더 중요한 지표가 될 수 있다.

100% 이전에도 성능 문제가 발생할 수 있다

시스템 자원 사용률이 반드시 100%에 도달해야만 성능 문제가 발생하는 것은 아니다.

자원 사용량이 증가하면서 요청이 대기하거나 내부 처리 지연이 발생할 수 있다.

따라서 자원 사용률이 100%에 도달한 뒤에야 문제를 탐지하는 방식은 적절하지 않을 수 있다.

Latency와 Saturation의 관계

Saturation이 증가하면 Latency가 함께 증가할 수 있다.

예를 들어 CPU가 처리 가능한 요청량에 가까워지면 새로운 요청이 대기하게 되고 처리 시간이 증가할 수 있다.

따라서 Latency의 증가는 시스템이 포화 상태에 접근하고 있다는 신호가 될 수 있다.

Saturation 예측

Saturation은 현재 자원 사용량뿐 아니라 가까운 미래에 자원이 부족해질 가능성도 포함한다.

예를 들어 디스크의 남은 공간과 증가 속도를 이용하여 몇 시간 후 공간이 부족해질지 예측할 수 있다.

6.5 Four Golden Signals 비교

Signal	핵심 질문	지표 예시
Latency	요청 처리에 얼마나 걸리는가?	응답 시간
Traffic	얼마나 많은 요청이 들어오는가?	Requests/sec
Errors	얼마나 많은 요청이 실패하는가?	Error Rate
Saturation	시스템 자원이 얼마나 소진되었는가?	CPU, Memory, Disk 사용률

이 네 가지 지표는 서로 연관되어 있다.

예를 들어 Traffic 증가로 인해 Saturation이 높아지면 Latency가 증가하고, 심한 경우 Errors가 발생할 수 있다.

다만 모든 시스템이 항상 동일한 방식으로 변화하는 것은 아니다.

따라서 서비스 특성에 맞게 각 지표를 해석해야 한다.

---

7. Worrying About Your Tail (or, Instrumentation and Performance)

서비스 성능을 측정할 때 평균값만 사용하는 것은 위험할 수 있다.

평균 응답 시간이 정상적이더라도 일부 요청이 매우 느리게 처리될 수 있기 때문이다.

이처럼 응답 시간 분포에서 극단적으로 느린 요청들을 **Tail Latency**라고 한다.

7.1 평균 Latency의 한계

예를 들어 웹 서비스의 평균 응답 시간이 100ms라고 가정한다.

그러나 일부 요청은 5초 이상 걸릴 수 있다.

평균값만 확인한다면 대부분의 요청이 어느 정도 빠르게 처리되는지 또는 일부 요청이 심각하게 느린지 구분하기 어렵다.

특히 하나의 사용자 요청이 여러 백엔드 서비스에 의존하는 경우, 일부 서비스의 긴 응답 시간이 전체 요청 처리 시간에 영향을 줄 수 있다.

7.2 Percentile

Percentile은 응답 시간의 분포를 파악하기 위한 지표이다.

대표적인 지표는 다음과 같다.

지표	의미
p50	요청의 50%가 해당 시간 이내에 완료
p95	요청의 95%가 해당 시간 이내에 완료
p99	요청의 99%가 해당 시간 이내에 완료

예를 들어 p99 Latency가 2초라면 전체 요청 중 약 99%가 2초 이내에 완료되었고, 약 1%는 그보다 오래 걸렸다는 의미이다.

p99는 일부 느린 요청이 사용자 경험에 미치는 영향을 파악하는 데 활용할 수 있다.

7.3 Histogram

Histogram은 요청 처리 시간을 여러 구간으로 나누어 기록하는 방식이다.

예를 들어 다음과 같이 측정할 수 있다.

응답 시간 구간	요청 수
0~10ms	150
10~30ms	300
30~100ms	400
100~300ms	120
300ms 초과	30

이 데이터는 예시이며, 각 구간의 요청 수를 이용하면 전체 응답 시간의 분포를 파악할 수 있다.

Histogram을 활용하면 평균값만으로 확인하기 어려운 느린 요청의 비율을 분석할 수 있다.

모든 요청의 응답 시간을 개별적으로 보관하지 않고 구간별 요청 수를 저장할 수 있다는 장점도 있다.

7.4 Tail Latency가 중요한 이유

분산 시스템에서는 하나의 사용자 요청이 여러 구성요소를 거칠 수 있다.

각 구성요소에서 발생하는 지연은 전체 사용자 요청의 처리 시간에 영향을 준다.

특히 여러 백엔드 서비스에 의존하는 경우, 일부 백엔드의 느린 요청이 사용자 관점에서 훨씬 큰 비율의 요청에 영향을 미칠 수 있다.

따라서 평균 응답 시간만으로 서비스 성능을 판단하지 않고 응답 시간의 분포를 함께 관찰해야 한다.

---

8. Choosing an Appropriate Resolution for Measurements

모니터링 데이터를 얼마나 자주 측정하고 수집할 것인지는 중요한 설계 요소이다.

모든 지표를 동일한 간격으로 측정할 필요는 없다.

서비스와 지표의 특성에 따라 적절한 측정 해상도(Resolution)를 선택해야 한다.

8.1 측정 간격이 중요한 이유

측정 간격이 너무 길면 짧은 시간에 발생한 이상 현상을 놓칠 수 있다.

반대로 측정 간격이 너무 짧으면 데이터 수집, 저장 및 분석에 필요한 비용이 증가한다.

따라서 관찰하려는 현상의 지속 시간과 측정 비용을 함께 고려해야 한다.

8.2 CPU 사용량 측정 예시

CPU 사용량을 1분 간격으로만 확인한다면 짧은 시간에 발생하는 CPU 사용량 급증을 놓칠 수 있다.

예를 들어 CPU가 몇 초 동안 과부하 상태에 들어갔다가 정상으로 돌아오는 상황에서는 1분 평균 사용률이 정상적으로 보일 수 있다.

이러한 현상은 Tail Latency 증가와 관련될 수 있다.

8.3 측정 데이터의 집계

짧은 간격으로 관찰해야 하지만 모든 원본 데이터를 장기간 저장할 필요는 없는 경우가 있다.

이때 내부에서 짧은 간격으로 데이터를 수집하고 외부 모니터링 시스템에는 집계된 데이터를 전달할 수 있다.

예를 들어 다음과 같은 방식을 사용할 수 있다.

1. 1초마다 CPU 사용량 측정
2. CPU 사용량을 5% 단위의 구간으로 분류
3. 각 구간에 속하는 측정 횟수 기록
4. 1분마다 구간별 기록을 집계

이 방식을 이용하면 짧은 시간의 CPU 사용량 증가를 관찰하면서 수집과 저장 비용을 줄일 수 있다.

측정 해상도는 단순히 높을수록 좋은 것이 아니라 관찰 목적과 비용에 맞게 선택해야 한다.

---

9. As Simple as Possible, No Simpler

모니터링 시스템은 필요한 기능을 제공하면서도 가능한 한 단순하게 유지되어야 한다.

지표, Dashboard, Alert 규칙을 계속 추가하다 보면 모니터링 시스템 자체가 복잡해질 수 있다.

예를 들어 다음과 같은 상황이 발생할 수 있다.

- 여러 지표마다 다른 Alert 기준 설정
- 다양한 Percentile에 서로 다른 Threshold 적용
- 모든 잠재적 원인을 탐지하기 위한 추가 규칙 작성
- 원인별 Dashboard를 계속 생성

이러한 기능이 지나치게 많아지면 Monitoring 시스템을 이해하고 변경하는 데 많은 시간이 필요해진다.

9.1 단순한 Alert 규칙

실제 장애를 탐지하는 핵심 Alert 규칙은 단순하고 예측 가능해야 한다.

운영자는 Alert가 발생했을 때 해당 Alert가 어떤 문제를 의미하는지 쉽게 이해할 수 있어야 한다.

9.2 불필요한 지표 제거

오랫동안 사용하지 않는 지표와 규칙은 제거를 검토할 수 있다.

예를 들어 다음과 같은 항목은 유지 필요성을 재검토할 수 있다.

- 실제 운영에서 거의 사용하지 않는 Alert 규칙
- Dashboard에 표시되지 않는 지표
- Alert에도 사용되지 않는 수집 데이터

불필요한 지표를 무조건 유지하면 모니터링 시스템의 관리 비용이 증가할 수 있다.

9.3 Monitoring과 Debugging 기능의 분리

Monitoring 시스템이 모든 분석 기능을 직접 제공할 필요는 없다.

예를 들어 다음 기능들은 모니터링과 관련이 있지만 별도의 도구로 구성할 수 있다.

- Detailed Profiling
- Log Analysis
- Traffic Inspection
- Load Testing
- Process Debugging

기본적인 Monitoring 시스템은 서비스 상태를 관찰하고 문제를 알리는 기능에 집중할 수 있다.

더 자세한 분석이 필요한 경우 전문적인 Debugging 도구와 연계하는 방식을 사용할 수 있다.

각 시스템을 느슨하게 연결하면 전체 모니터링 구조의 복잡성을 줄일 수 있다.

---

10. Tying These Principles Together

앞서 설명한 원칙들은 효과적인 Monitoring 및 Alerting 시스템을 설계하기 위한 기준으로 연결된다.

특히 중요한 것은 **어떤 상황에서 사람에게 Page를 보낼 것인가**이다.

운영자를 호출하는 Alert는 긴급하고 실제 대응이 필요한 문제를 대상으로 해야 한다.

10.1 Alert 규칙을 설계할 때 확인할 질문

새로운 Alert 규칙을 만들 때 다음과 같은 질문을 검토할 수 있다.

① 이 Alert는 긴급하고 대응 가능한 문제를 탐지하는가?

사용자에게 이미 영향을 주고 있거나 곧 영향을 줄 가능성이 높은 문제인지 확인한다.

② 이 Alert를 무시해도 되는 상황이 존재하는가?

정상적인 운영 상황에서도 발생하는 Alert라면 불필요한 호출이 반복될 수 있다.

③ 실제 사용자에게 영향을 주는 문제인가?

내부 지표가 비정상적으로 보이더라도 사용자 서비스에 문제가 발생하지 않을 수 있다.

④ Alert를 받은 운영자가 수행할 작업이 존재하는가?

대응 방법이 없다면 즉시 운영자를 호출하는 것이 적절한지 검토해야 한다.

⑤ 지금 대응해야 하는가?

즉시 대응이 필요한 문제와 다음 업무 시간에 처리해도 되는 문제는 구분해야 한다.

⑥ 자동화할 수 있는 작업인가?

항상 동일한 방식으로 대응할 수 있는 문제라면 자동화를 고려해야 한다.

⑦ 중복된 Alert가 발생하지 않는가?

하나의 문제로 여러 운영자가 불필요하게 동시에 호출되는 상황을 줄여야 한다.

10.2 좋은 Page의 조건

효과적인 Page는 다음과 같은 특성을 가져야 한다.

- 긴급한 대응이 필요하다.
- 운영자가 실제로 수행할 수 있는 작업이 있다.
- 단순히 정해진 명령을 반복하는 것 이상의 판단이 필요하다.
- 반복적으로 발생하는 동일한 수동 대응을 요구하지 않는다.

Page가 자주 발생하면 운영자의 집중력과 대응 능력이 저하될 수 있다.

따라서 운영자의 주의를 요구하는 Alert는 신중하게 설계해야 한다.

10.3 증상 중심의 Alert

즉각적인 대응을 요구하는 Alert에서는 실제 사용자에게 나타나는 증상을 중심으로 판단하는 것이 중요하다.

서비스 내부의 모든 이상 징후가 Page를 발생시켜야 하는 것은 아니다.

반면 사용자에게 영향을 주는 장애가 진행 중이거나, 매우 확실하고 임박한 장애 원인이 존재하는 경우에는 Page가 필요할 수 있다.

결국 Alert의 가치는 어떤 Monitoring 방식에서 발생했는지가 아니라, 실제로 대응할 가치가 있는 문제를 정확하게 알려주는지에 따라 결정된다.

11. Monitoring for the Long Term

모니터링 시스템은 현재 발생한 장애를 탐지하는 것뿐 아니라 장기적으로 서비스를 안정적으로 운영할 수 있도록 설계되어야 한다.

시스템의 구조, 트래픽, 성능 목표는 시간이 지나면서 변화한다.

따라서 Alert 기준과 대응 방식도 이에 맞게 개선해야 한다.

11.1 반복적인 Alert의 문제

처음에는 드물게 발생하던 장애가 시간이 지나면서 자주 발생할 수 있다.

이러한 문제에 대해 운영자가 항상 같은 대응을 반복한다면 운영 부담이 증가한다.

이 경우에는 단순히 Alert를 유지하는 것이 아니라 반복적인 문제의 Root Cause를 해결할 필요가 있다.

근본 원인을 즉시 해결하기 어려운 경우에는 임시 대응을 자동화할 수 있다.

단, 임시 자동화가 근본적인 문제 해결을 무기한 미루는 이유가 되어서는 안 된다.

11.2 Bigtable SRE: A Tale of Over-Alerting

Bigtable에서는 서비스 성능 목표와 관련된 Alert가 너무 많이 발생하는 문제가 있었다.

당시 Bigtable의 성능에는 일부 매우 느린 요청이 큰 영향을 주고 있었다.

이러한 성능 문제로 인해 많은 Email Alert와 Page가 발생했다.

문제점

- 지나치게 많은 Alert 발생
- 실제 대응이 필요하지 않은 Alert 존재
- 운영자의 Alert 분석 부담 증가
- 중요한 사용자 영향 장애를 놓치는 상황 발생
- 장기적인 성능 개선에 사용할 시간 부족

운영자는 반복적으로 발생하는 Alert를 확인하고 대응하는 데 많은 시간을 사용해야 했다.

개선 방향

Bigtable SRE 팀은 장기적인 성능 문제를 해결하기 위해 다음과 같은 조치를 취했다.

1. Bigtable과 하부 스토리지 시스템의 성능 개선

2. 일시적으로 SLO 측정 및 목표를 완화하여 75번째 백분위수 Latency를 사용
3. 과도한 Email Alert 비활성화

이 조치는 75번째 백분위수를 일반적인 SLO 기준으로 권장한다는 의미가 아니다.

과도한 Alert로 인해 근본적인 개선 작업이 방해받는 상황에서 일시적으로 운영 부담을 낮추기 위한 선택이었다.

불필요한 Alert를 줄이자 운영자는 장기적인 문제를 해결하는 데 더 많은 시간을 사용할 수 있었다.

핵심

모든 문제에 대해 Alert를 발생시키는 것이 아니라, 실제로 대응이 필요한 문제를 정확하게 알리는 것이 중요하다.

11.3 Gmail: Predictable, Scriptable Responses from Humans

초기 Gmail에서는 분산 프로세스 관리 시스템인 Workqueue를 사용했다.

이 시스템에서 작업의 스케줄링 문제가 발생하면 Monitoring 시스템이 Alert를 발생시켰다.

그러나 Gmail은 매우 많은 작업으로 구성되어 있었기 때문에 개별 작업에 대해 Alert를 발생시키는 방식은 운영하기 어려웠다.

문제점

- 개별 작업 단위의 많은 Alert
- 반복적인 작업 재스케줄링 문제
- 운영자의 수동 개입 필요
- 동일한 문제에 대한 정형화된 대응 반복

Gmail SRE 팀은 스케줄러 문제의 영향을 완화하기 위한 도구를 개발했다.

이 도구를 이용하면 운영자가 반복적으로 수행하던 대응을 보다 쉽게 처리할 수 있었다.

자동화에 대한 고민

문제 탐지부터 복구 작업까지 전체 과정을 자동화할 수 있었지만, 임시 대응을 자동화하면 근본 원인을 해결하는 작업이 뒤로 밀릴 수 있다는 우려도 있었다.

그러나 동일한 수동 대응이 반복되는 상태를 유지하는 것도 운영 부담을 증가시킨다.

따라서 반복 작업은 자동화하면서도 근본적인 문제 해결을 별도로 추진해야 한다.

핵심

운영자가 항상 동일한 절차를 반복하고 있다면 자동화할 수 있는 작업인지 검토해야 한다.

자동화는 운영 부담을 줄일 수 있지만 Root Cause 해결을 대체하지는 않는다.

11.4 The Long Run

Bigtable과 Gmail 사례는 단기적인 가용성과 장기적인 안정성 사이에 균형이 필요하다는 점을 보여준다.

운영자가 반복적인 수동 대응을 수행하여 단기적으로 서비스를 유지할 수는 있다.

그러나 이러한 방식이 지속되면 운영 부담과 기술 부채가 증가할 수 있다.

장기적인 안정성을 높이기 위해서는 다음 사항을 고려해야 한다.

- 반복적인 장애의 Root Cause 해결
- 자동화 가능한 대응의 자동화
- 불필요한 Alert 감소
- 운영자의 대응 부담 관리
- 지속적으로 개선 가능한 시스템 설계

단기적인 운영 편의만을 고려하지 않고 장기적으로 유지할 수 있는 Monitoring과 Alerting 구조를 만들어야 한다.

---

12. Conclusion

효과적인 Monitoring 및 Alerting 시스템은 단순하고 이해하기 쉬워야 한다.

Monitoring은 서비스의 상태를 파악하고 문제를 분석하기 위해 다양한 데이터를 수집한다.

그러나 모든 데이터를 즉각적인 Alert로 연결할 필요는 없다.

12.1 Symptom 중심의 Monitoring

운영자가 즉시 대응해야 하는 문제는 실제 서비스에 나타나는 증상을 중심으로 탐지하는 것이 중요하다.

내부 지표는 주로 문제의 원인을 조사하거나 장애가 발생하기 전에 문제를 파악하는 데 활용할 수 있다.

12.2 단순한 Alert 구조

Alert는 명확한 문제를 나타내고 운영자가 수행할 수 있는 조치와 연결되어야 한다.

지나치게 많은 Alert는 실제 중요한 문제를 발견하기 어렵게 만들 수 있다.

12.3 Dashboard와 Debugging

긴급하지 않은 문제는 Dashboard 등을 통해 지속적으로 관찰할 수 있다.

문제가 발생했을 때는 로그 및 내부 지표와 함께 과거 데이터를 비교하여 원인을 분석한다.

12.4 장기적인 개선

모니터링 시스템은 시스템의 변화에 맞게 지속적으로 개선해야 한다.

불필요한 Alert를 제거하고 반복적인 대응을 자동화하며 근본적인 원인을 해결하는 것이 중요하다.

---

13. OpenStack 네트워크 스터디에 적용하기

이 절은 SRE 6장의 원칙을 OpenStack 네트워크 운영 학습에 적용한 예시이며, 책의 원문 내용을 직접 요약한 부분과 구분한다.

OpenStack 네트워크는 여러 서비스와 네트워크 구성요소가 함께 동작하는 시스템이다.

따라서 개별 구성요소의 상태와 실제 VM의 네트워크 연결 상태를 함께 확인할 필요가 있다.

13.1 Black-box와 White-box 적용

예를 들어 VM에서 외부 네트워크로 연결이 가능한지 확인한다고 가정한다.

Black-box Monitoring

- VM에서 외부로 ICMP 요청 전송
- VM에서 외부 HTTP 서비스 접근
- 외부에서 VM의 Floating IP로 TCP 연결 시도

이러한 확인은 실제 네트워크 경로의 사용 가능 여부를 관찰한다.

White-box Monitoring

- Neutron Agent 상태
- OVS Bridge 및 Port 상태
- 네트워크 인터페이스 통계
- Router 설정

- 패킷 처리 및 오류 카운터

이러한 확인은 네트워크 내부 구성요소의 상태를 파악하는 데 활용된다.

예시

```
VM → External HTTP 요청
            |
            v
       요청 실패
            |
            v
Black-box: 외부 연결 실패 확인
            |
            v
White-box: 내부 네트워크 구성 확인
            |
            +--- VM Interface
            |
            +--- Virtual Switch
            |
            +--- Router
            |
            +--- External Network
```

Black-box Monitoring을 통해 통신 실패를 확인하고 White-box Monitoring을 이용하여 어느 구성요소에서 문제가 발생했는지 조사할 수 있다.

13.2 Four Golden Signals 적용

OpenStack 네트워크에도 Four Golden Signals를 적용할 수 있다.

Signal	OpenStack 네트워크 적용 예시
Latency	VM 간 RTT, TCP 연결 시간, HTTP 응답 시간
Traffic	네트워크 인터페이스 패킷 수, 처리량
Errors	연결 실패율, 패킷 손실, 요청 실패율
Saturation	인터페이스 사용률, Conntrack Table 사용률

이 지표들은 관찰 목적에 따라 선택해야 한다.

예를 들어 VM 간 연결 실패가 주요 관심사라면 인터페이스의 전체 처리량보다 실제 VM 간 연결 성공 여부가 더 직접적인 지표가 될 수 있다.

또한 패킷 손실이나 지연은 서비스에 따라 서로 다른 영향을 줄 수 있으므로 사용자 관점의 정상 동작 기준을 함께 고려해야 한다.

13.3 Symptom과 Cause 적용

예를 들어 다음과 같은 사용자 신고가 발생했다고 가정한다.

> VM에서 외부 네트워크로 접속할 수 없습니다.

이것은 Symptom이다.

가능한 Cause는 다음과 같다.

- VM 네트워크 연결 문제
- Virtual Switch 연결 문제
- Router 설정 문제
- NAT 문제
- Security Group 정책 문제
- External Network 연결 문제

사용자 신고만으로 특정 원인을 확정할 수는 없다.

따라서 실제 연결을 확인하고, 관찰한 결과를 바탕으로 원인 후보를 줄여나가는 방식으로 조사해야 한다.

13.4 정상 상태의 기준선(Baseline)

장애를 분석하기 전에 정상 상태를 관찰하고 기록하면 이후 문제가 발생했을 때 비교 기준으로 활용할 수 있다.

예를 들어 다음 항목을 기록할 수 있다.

관찰 대상	정상 상태 확인 항목
Neutron	필요한 Network, Port, Router 존재
VM	IP 주소와 네트워크 설정
VM 간 통신	연결 성공 여부
외부 통신	HTTP 또는 ICMP 요청 성공 여부
Virtual Switch	필요한 Bridge 및 Port 연결
네트워크 지표	정상 상태의 지연 및 오류 수준

정상 상태를 기록할 때는 단순히 성공 또는 실패 여부뿐 아니라 어떤 조건에서 어떤 방식으로 관찰했는지도 함께 남기는 것이 좋다.

예를 들어 작은 ICMP 패킷이 정상적으로 전달되었다는 사실만으로 모든 크기의 패킷이나 TCP 연결이 정상이라고 결론 내릴 수는 없다.

관찰한 결과가 어떤 범위의 정상 동작을 확인하는지 명확히 해야 한다.

13.5 SRE 관점의 핵심 질문

매번 네트워크 장애를 분석할 때 다음 질문을 적용할 수 있다.

1. 사용자가 실제로 경험한 문제는 무엇인가?
2. 문제를 확인할 수 있는 Black-box 지표는 무엇인가?
3. 장애 원인을 조사하기 위해 필요한 White-box 지표는 무엇인가?
4. 어떤 관찰 결과를 통해 원인 후보를 줄일 수 있는가?
5. 사용자가 신고하기 전에 문제를 감지할 수 있는 방법이 있었는가?
6. 이 문제가 발생했을 때 즉시 운영자에게 Alert를 보내야 하는가?
7. 같은 장애가 반복된다면 어떤 대응을 자동화할 수 있는가?

---

14. Chapter 6 종합 정리

핵심 개념	설명
Monitoring	시스템 데이터를 수집·처리·집계·표시
Alerting	사람이 대응해야 할 문제를 통지
Symptom	서비스에 나타난 문제의 현상
Cause	문제를 발생시킨 원인
Black-box	사용자 관점의 외부 동작 확인
White-box	내부 지표 및 상태 관찰
Latency	요청 처리 시간
Traffic	요청 또는 작업의 양
Errors	실패한 요청의 비율
Saturation	시스템 자원 사용 및 포화 수준
Tail Latency	일부 느린 요청이 나타나는 응답 시간 분포의 끝부분
Measurement Resolution	지표를 측정하는 시간적 세밀함
Root Cause	장애가 발생하게 된 근본적인 결함이나 원인
Actionable Alert	운영자가 실제로 대응할 수 있는 문제를 알리는 Alert

핵심은 가능한 많은 지표를 수집하는 것이 아니라 서비스에 발생한 문제를 빠르게 발견하고, 실제로 필요한 대응을 할 수 있도록 모니터링 시스템을 설계하는 것이다.

---

15. 학습 확인 질문

1. Monitoring과 Alerting은 어떤 차이가 있는가?
2. Dashboard와 Page는 각각 어떤 목적을 가지는가?
3. Symptom과 Cause는 어떻게 구분할 수 있는가?
4. Black-box Monitoring과 White-box Monitoring의 차이는 무엇인가?
5. 두 Monitoring 방식을 함께 사용하는 이유는 무엇인가?
6. Four Golden Signals는 무엇이며 각각 어떤 의미를 가지는가?
7. Latency를 측정할 때 성공한 요청과 실패한 요청을 구분해야 하는 이유는 무엇인가?
8. 평균 Latency만 확인하면 어떤 문제를 놓칠 수 있는가?
9. Percentile과 Histogram을 사용하는 이유는 무엇인가?
10. 측정 간격을 지나치게 짧게 설정하면 어떤 문제가 발생할 수 있는가?
11. 모든 비정상적인 지표에 대해 Page를 발생시키면 안 되는 이유는 무엇인가?
12. Bigtable 사례에서 Alert를 일시적으로 줄인 이유는 무엇인가?
13. Gmail 사례에서는 반복적인 운영 대응과 자동화 사이에서 어떤 문제를 고민했는가?
14. 운영자가 항상 같은 절차로 복구하는 장애가 반복된다면 장기적으로 어떤 개선을 고려해야 하는가?
15. OpenStack에서 Neutron Agent가 정상 상태인데 실제 VM 통신이 실패한다면 어떤 Monitoring 방식으로 문제를 확인할 수 있는가?

최종 학습 목표: 사용자 관점의 장애와 내부 원인을 구분하고, Black-box/White-box Monitoring과 Four Golden Signals를 이용하여 의미 있는 관찰 지표 및 Alert 기준을 설명할 수 있어야 한다.